Open Architecture Handbook

The Borland Developer's Technical Guide

BORLAND INTERNATIONAL, INC.
100 BORLAND WAY
P.O. BOX 660001
SCOTTS VALLEY, CA 95067-0001

implementation of a windows system. Other brand

and product names are trademarks or registered trademarks of their respective holders.

PRINTED IN THE USA.
R1 10 9 8 7 6 5 4 3 2 1

INTRODUCTION

This book presents technical information about several of Borland's language

tools, including

internal functions

implementation details

file formats, and

other technical specifications

It is for advanced users and corporate developers who want to utilize the

"behind the scenes" features of Borland's products to develop their own

customized tools and environments, and to provide better compatibility with

existing code and tools from other vendors.

Why open architecture?

At the beginning the PC's second decade, one word has captured the spirit and attention of the entire computer industry.

It is the word open.

Today we hear more and more about open systems, open standards, open tools . . . and open architectures. Along with object-oriented design, the open architecture movement heralds a new era of modular software that is designed to be shareable, extensible and compatible.

Just as today's users want a database that integrates smoothly with their word processor, their spreadsheet and their company's mainframe database, so today's software developers demand editors, compilers, debuggers, application frameworks and other tools that they can "mix and match," tools that they can extend or enhance themselves, open development environments that they can customize to work the way they want.

The age of closed environments and inaccessible proprietary architectures is coming to an end. With more open and compatible software tools, programmers are better able to create the exciting, reliable and cost-effective software that the nineties will demand.

Borland language tools

As a leader in the object-oriented design revolution, Borland maintains an unqualified commitment to the open architecture movement. This book, as part of that commitment, provides detailed technical information about the "guts" of Borland's language development tools: internal file formats, compiler implementation details, debugger record structures and much more.

This information will enable programmers to extend Borland's tools to meet their own needs, help third-party developers spawn compatible add-on tools, aid software engineers in squeezing out the utmost performance levels from their code by taking advantage of implementation-specific features, and give all programmers greater control and independence over their development environment.

How to use this book

This book, as befits its subject, is not for the novice user or the technically unsophisticated. Written largely by the Borland developers who actually created the tools described, its style is terse and technical. Every effort has been made to present the topics clearly and in an easy-to-read manner, but the presentation is not a "tutorial," nor are basic concepts of the tools discussed at great length. It is best viewed as a collection of technical papers by developers for developers, presenting hard to find information in a convenient and readily-accessible form.

In the chapters which follow, individual specifications will be presented for these Borland tools and standards:

Tools discussed

C++ object mapping: a detailed description of the Borland C++ implementation's internal strategy for representing objects of various types. Included are the compiler's name mangling rules and discussions of class datas and function members, object initialization, hidden parameters, RTL helper functions, virtual tables and vtable pointers, and dynamically dispatched virtual tables (DDVTs).

Object file format: a listing of the structure and content of each type of record emitted by Borland C++ when it produces object files.

VIRDEF records: a discussion and format listing of Borland's VIRDEF record type. VIRDEF records are utilized by the linker to support virtual definitions for some C++ types. A VIRDEF record is otherwise similar to a COMDEF record.

Symbol table format: presents a brief discussion and layout of the general symbol table which appears at the head of each .EXE file. The symbol table contains TLINK debugger and browser information.

Project file format: a detailed layout of the Borland C++ Project file format, used by IDE's Project Make facility.

Borland Graphics Interface: describes BGI driver architecture, headers, status and vector tables, structure, and provides a cookbook and examples.

ObjectWindows: Borland's ObjectWindows Library (OWL) is a complete application framework for Windows developers. This chapter presents the technical specification for the library, including class structure, protocol and behavior, as well as implementation notes.

Borland Windows Custom Controls: presents the technical specifications, usage conventions, and a listing of notification messages in the BWCC custom controls and dialog classes.

Borland Help System: defines the Borland Help System, including the source text file format, binary Help file format, and the run-time Help engine.

Accompanying software

The accompanying Examples and Supplementary Software disk contains a number of brief example programs utilizing the information contained in this book. The examples are referenced in the chapter(s) to which each example applies.

A brief disclaimer

The information presented in this guide is for the benefit of advanced developers who wish to take advantage of various internal features and formats

of the Borland tools.

We hope this information is helpful to you and enhances the usefulness of Borland's language development products. Due to its highly technical nature, this material is not documented in the product manuals, and cannot be supported by our customer service staff.

CHAPTER

1

C++ object mapping

This chapter describes how Turbo C++ and Borland C++ handles memory for C++ objects.

The following applies both to the 16-bit (segmented address space) and the 32-bit versions of BCC. Whenever the text has a near or far pointer, this applies to the 16-bit version, and a 32-bit (flat) pointer is to be substituted for the 32-bit version. When the text describes two near and far flavors of the same data structure, a single version using 32-bit flat pointers is to be used for the 32-bit product.

Nonstatic data members

Borland C++ compilers allocate space for nonstatic data members in order of declaration and regardless of access specifiers. When the word alignment compiler option is turned on, all members larger than 1 byte are aligned on a word boundary (the 32-bit compiler allows alignment on both a multiple of 2 and a multiple of 4 offset, depending on the state of the alignment options and/or the presence of #pragma pack).

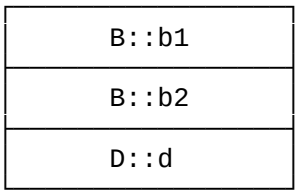
Nonvirtual base classes

Nonvirtual base class members, including compiler defined members, such as vtable pointers, always precede any derived class members, and are allocated in order of declaration, as shown in the following example. Padding is inserted if dictated by the state of compiler alignment options.

```
class B
```

```
{
  int b1, b2;
};

class D:B
{
  int d;
};
```

The following diagram represents an instance of D:



Virtual base classes

At the point where a particular base would occur in an object if the base weren't virtual, which is the case in the previous example, a virtual base class pointer is stored instead, and all of the virtual bases for an object follow all of the nonvirtual bases as well as the derived class, in the order of construction as specified by the language.

The virtual base class pointer is always a 16-bit offset pointer, because a class instance can't span a segment boundary. A compiler option offered for backward compatibility with previous releases of Turbo C++ and Borland C++ allows the virtual base class pointer to be either a near or a far pointer,

depending on size of the this pointer for that class (16-bit compiler only; with the 32-bit compiler, the virtual base pointers are always flat 32-bit pointers).

The compiler will insert a hidden 'unsigned int' (i.e. 16-bit for the 16-bit compiler, 32-bit for the 32-bit compiler) displacement member immediately preceding the virtual base class sub-object when the following conditions exist:

a class has either a user-defined constructor or destructor, or both

the derived class overrides a virtual function defined in one if its virtual bases

The displacement member always equals zero with the following exception, which occurs during construction/destruction of the derived class object: If the derived object is embedded in another class and the virtual base class isn't at the same offset from the derived class as it would be in an object of the derived class, then the displacement member is nonzero. The nonzero displacement member is then used in virtual function thunks to ensure that a correct value is passed to the virtual function for the this parameter. For compatibility with older versions of Turbo C++ and Borland C++, a compiler option disables the addition of the hidden displacement member on a per-class basis.

The compiler ensures that the derived class of a virtual base with another virtual base has the 'indirect' virtual base as its virtual base for the following reasons:

to represent member pointers capable of pointing to members of virtual base classes in a compact and efficient way

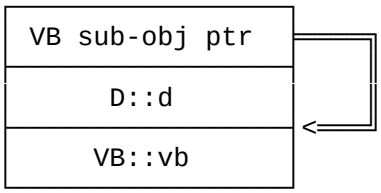
to limit the involvement of 'derived*' to 'base*' casts to just one virtual base class pointer indirection

The compiler adds such virtual base classes following any user-specified base classes, in the order of construction, but the addition occurs only when the particular virtual base can't already be reached from the derived class through only one level of virtual inheritance. The presence of compiler-added virtual base classes doesn't have side-effects such as changing visibility rules. A compiler-added virtual base class is used for casts of pointers and for pointers of members of the virtual base. The representation of member pointers is discussed later.

The following example shows the declaration of the simplest virtual base:

```
class VB
{
  int vb;
};

class D:virtual VB
{
  int d;
};
```

The instance of D has the following layout:



The following example shows the declaration of an indirect (or doubly) virtual base:

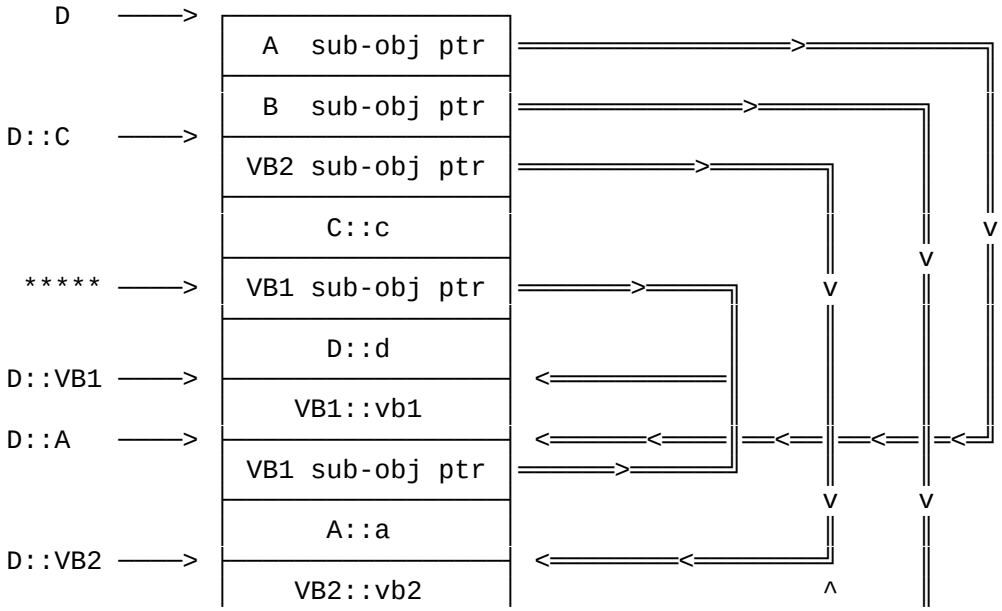
```
class VB1
{
  int vb1;
};

class VB2
{
```

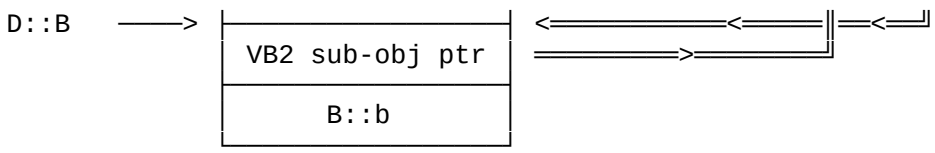
```
  int vb2;
};

class A:virtual VB1
{
  int a;
};

class B:virtual VB2
```



```
{
  int b;
};

class C:virtual VB2
{
  int c;
};

class D:virtual A, virtual B, C
{
  int d;
};
```

An instance of D has the following layout:





The virtual base VB2 is reachable from D through only one level of inheritance due to the base class C; therefore, VB2 isn't added by the compiler as a virtual base of D. The diagram shows the VB1 base pointer (see *****), which is added by the compiler.

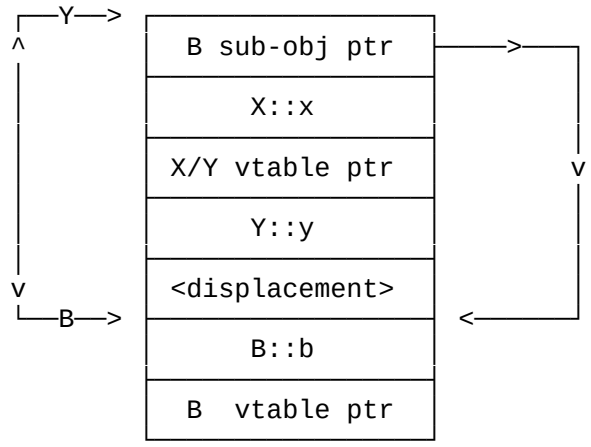
The following example shows a hidden displacement member:

```
class B
{
```

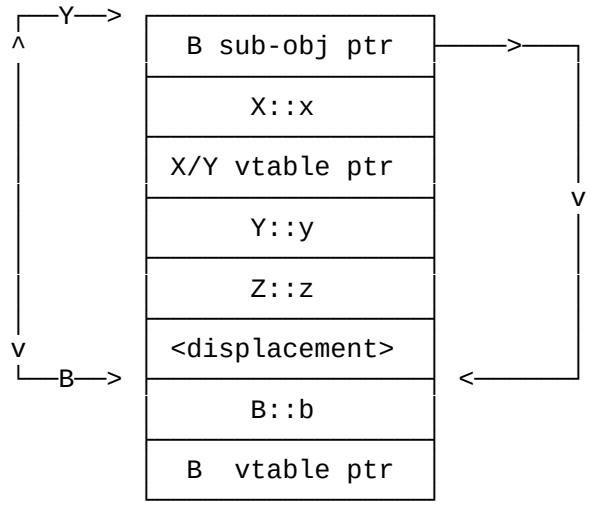


```
  B();
virtual void    f();
  int b;
};

class   X:virtual B
{
  int x;
};

class   Y:X
{
  Y();

virtual void    f();

  int y;
};

class   Z:Y
{
  int z;
};
```

An instance of Y has the following layout:



An instance of Z has the following layout:



As shown by the diagrams, the displacement between the B sub-object and the base of Y differs by 2 bytes, depending on whether the object is of type Y or Z. The displacement member will be set to -2 in the constructor Z::Z before X::X is called. The displacement member is reset to zero after the call to X::X has been completed. The virtual table thunk for the Y::f entry in the B part of X/Y's vtable will adjust the value of this by the current value of the displacement member, which is zero unless the current object is being constructed or destructed.

Empty classes

A class without any nonstatic data members is allocated 1 or 2 bytes (1, 2, or 4 bytes with the 32-bit compiler), depending on alignment options selected. Exception: if the class has virtual functions, the instance simply consists of the vtable pointer, and no padding is added.

Addressing of class instances and this

The address of a class instance is always the first byte allocated. For derived classes, the address is typically the first member of the 'root' class (the base-most base class).

The size of the this pointer defaults to the default pointer size for the memory model in effect. Declaring the class itself as near or far overrides this default. A derived class inherits the size of this from the first base, and all of the following bases (if any) must use the same this size.

Virtual table pointers

When a vtable pointer is introduced in a class, it's inserted before any user-defined members of that class and after any base class sub-objects; a compiler option forces the vtable pointer member to be added after all user-defined

members of the class, allowing many C++ structures with virtual function members to be easily shared with other languages, such as C.

In the huge memory model, the vtable pointer is always far, while in all other memory models, the vtable pointer defaults to near. Declaring a class as huge or _export has the following consequences:

overrides the default, making the vtable pointer far

allocates the vtable either in the code segment or in a data segment specified by compiler options

A vtable pointer in a derived class is shared with the first base if the base is nonvirtual and if it already contains a vtable pointer.

Virtual tables

A virtual table is a table of function pointers; near and far pointers can be arbitrarily mixed. No padding is added to align the far pointers.

Virtual function calls, virtual thunks

When a virtual function is called using the virtual mechanism, the value passed for this always points to the appropriate sub-object by the time execution arrives at the virtual function. When multiple inheritance is involved, any virtual functions inherited from a virtual base (or from a base that isn't the first base) and overridden in the derived class are dispatched through a virtual thunk. The pointer in the virtual table points to this thunk, which then adjusts the this value on the stack and jumps to the function body itself.

A virtual thunk in a virtual base's vtable adds the value of the hidden 16-bit (32-bit for the 32-bit compiler) displacement member (described previously) to the this value under the following condition: a virtual function overrides a virtual in a virtual base class containing one or more user-defined constructors or a user-defined destructor.

This technique ensures passage of the correct this value to the function regardless of the relative distance between the derived class and the virtual base. The relative distance might be different than the distance between the derived class and the 'pure' derived instance.

Calling conventions for member functions

The default calling convention for member functions is cdecl with user arguments pushed right-to-left, followed by the this pointer; the caller pops the arguments from the stack. With the pascal calling convention, any user arguments are pushed first from left to right, this is again pushed last, and the callee pops the arguments from the stack. A compiler option is available that passes this as the first argument to pascal member functions (for compatibility with other compilers and previous versions of Turbo C++ and Borland C++).

Pointers to class members

There are three general categories of member pointers: single inheritance (SI), multiple inheritance without virtual bases (MI), and the most general (VB). See the Borland C++ User's Guide for more information on how the compiler (or the user) chooses the effective representation for a specific member pointer type.

Sometimes when a VB member pointer, which is capable of pointing to members of virtual bases, is cast to another member pointer type, the cast can't be carried out using inline code, and the compiler generates a call to an RTL helper function, which is discussed in more detail later.

Note that for all categories of member pointers, a NULL pointer value has all the fields of the member pointer equal to zero. When testing a member pointer value for NULL, the compiler might test only some fields of the member pointer.

Pointers to data members

The following internal representation of pointers to data members describe two categories of pointers: unrestricted pointers, which can point to any member of any class, and pointers that can't point to members of virtual base classes:

SI/MI data member pointer

```
size_t member_offset;
```

The SI/MI data member pointer is an offset within the class instance of the member being pointed to with one added to it, allowing zero to be used as a NULL pointer.

VB data member pointer

```
size_t member_offset;
size_t vbcptr_offset;
```

The VB data member pointer consists of two offsets; if vbcptr_offset is nonzero, the pointer points to a virtual base class member, and vbcptr_offset gives the offset of the virtual base class pointer within the object plus one. member_offset then specifies the offset within that virtual base class to the member being pointed to. When vbcptr_offset is zero, the pointer is treated just like the "SI/MI" data member pointer.

Pointers to function members

Pointers to member functions resemble pointers to data members, except they always contain a function pointer. If the function is nonvirtual, the function pointer either points to the member function or to a virtual call thunk that uses the virtual mechanism to transfer control to an appropriate virtual function. The compiler creates such thunks automatically. When calling through a pointer to a member function, the appropriate value must be passed to the function for the this parameter. When calling using the ->* operator, the value equals the object pointer, while when calling using the .* operator, the value equals the address of the object. The address must be adjusted based on additional fields in the member pointer:

SI function member pointer

```
void (*func_addr)();
```

The SI function member pointer contains the address of the member function. The function pointer is appropriately typed, based on the member pointer type.

MI function member pointer

```
void (*func_addr)();
size_t member_offset;
```

The MI function member pointer adjusts the this value passed to the member function by member_offset - 1.

VB function member pointer

```
void (*func_addr)();
size_t member_offset;
size_t vbcptr_offset;
```

The VB function member pointer adjusts the this value passed to the member function with an algorithm similar to the one that adjusts offsets for VB data member pointers.

Static data members

Static data members default to near in all memory models except the huge model; however, static data members of classes declared _export always default to far in all memory models.

__export/__import classes

Declaring a class __export causes all of its noninline member functions and static data members to be exported, it also makes the vtable pointer far, and allocates the virtual table for the class in the code segment; moreover, declaring a class __export or __import causes all of the static data members and member functions of the class to default to far.

Passing classes by value

When a function accepts a class with constructors argument, the actual argument value is copy-constructed onto its place on the stack, and the called routine calls the destructor for the argument if its class has a destructor. A compiler

option causes the compiler to convert class with constructors arguments to reference to class arguments, and the compiler creates temporary storage at the calling site to hold the argument value (for compatibility with older versions of Turbo C++ and Borland C++).

Initialization and finalization of nonlocal static objects

The compiler initializes and finalizes nonlocal static objects in each compilation as required. The functions included for initialization and finalization are registered through the standard Turbo C++ and Borland C++ #pragma startup/exit mechanism.

Conventions for constructors and destructors

When the compiler passes a hidden parameter in addition to this, for example, when calling a constructor, the parameter is passed as if it were to the right of this and to the left of the first user argument if such an argument exists.

Constructors

The compiler passes a constructor the address of object memory to be constructed, or it passes a zero for this, in which case the constructor allocates the memory for the object through the operator new. If the allocation fails, the constructor immediately returns zero; in all other cases, the constructor returns the address of the object constructed.

The compiler gives constructors for classes with any virtual bases (direct or indirect) an extra int parameter to indicate the following action:

A zero means the constructor should construct all virtual base classes. (The class is known to be the most-derived class, and the location of all virtual bases within the object is known at compile time.)

A nonzero means virtual bases have already been constructed by a derived class constructor.

Destructors

A destructor tests this for NULL before taking other action on an object. If this is NULL, the destructor immediately returns.

All destructors are passed an extra int parameter that contains two bit flags:

0x01 When this bit is on, the destructor calls operator delete to deallocate the memory taken up by the object, then the destructor returns.

0x02 When this bit is on, all virtual bases are destroyed. This bit is only used for classes with virtual bases.

RTL helper functions

The run-time library supplies several helper functions to the compiler for allocating, deleting, and copying certain arrays of classes.

The following functions, _vector_apply_ and _vector_applyv_, have "C" linkage.

```
extern "C"
void  _vector_apply_
        (
        void far *  dest,  // address of destination array
        void far *  src,   // address of source array
        size_t size,       // size of each object
        unsigned count,    // number of objects
        unsigned mode,     // type of function to call
        ...                // operator=/copy-constructor address here
        )

extern "C"
void  _vector_applyv_
        (
        void far * dest,
        void far * src,
        size_t size,
        unsigned count,
        unsigned mode,
        ...
        )
```

_vector_apply_ and _vector_vapply_ assign or copy-construct class elements of the type array of class type. Since the operator= or the copy-constructor might be a near or far function, and take a near or far this value, mode is passed to determine how to cast this. A near pointer must be passed for near functions and a far pointer for far functions, and it's impossible to determine the argument type until runtime; consequently, varargs is used to resolve the problem. The compiler guarantees that source and destination are both near or both far.

The version with the v suffix passes a second argument of zero for copy-constructors of classes with virtual bases.

The following list shows the interpretation of the mode for _vector_apply_ and _vector_vapply_:

far function	0x01
pascal call	0x02
far pointer	0x04

The following functions, _vector_new_ and _vector_vnew_, which return near pointers to void, have C++ linkage. They are used only in the tiny, small, and medium memory models:

```
void near * _vector_new_
        (
        void    near *  ptr,    // address of array (0 means allocate)
        size_t          size,   // size of each object
        unsigned        count,  // how many objects
        unsigned        mode,   // mode bits (see below)
        ...                     // constructor address passed here
        );

void near * _vector_vnew_
        (
        void    near *  ptr,
```

```
        size_t          size,
        unsigned        count,
        unsigned        mode,
        ...
        );
```

The following functions, which return far pointers to void, exist in all memory models:

```
  void  far * _vector_new_
        (
        void     far *  ptr,
        size_t          size,
        unsigned long   count,
        unsigned        mode,
        ...
        );

  void  far * _vector_vnew_
        (
        void     far *  ptr,
        size_t          size,
        unsigned long   count,
        unsigned        mode,
        ...
        );
```


The following list shows the interpretation of the mode for _vector_new and _vector_vnew:

```
  far function              0x01
  pascal call               0x02
  far pointer               0x04
  store element count       0x10
  huge array (array > 64K)  0x40
```

The _vector_new_ and _vector_vnew_ routines construct arrays of class type. If ptr is NULL, the routines allocate the space for the array. If mode has 0x10 set, allocated space includes a count field stored at the beginning. If mode has 0x40 set, the pointer returned must be adjusted to prevent a class from crossing

the 64K boundary, and the address passed back is adjusted accordingly. Since the constructor for the class might be a near or a far function, and take a near or far this value, mode is passed to allow correct casting. A near pointer must be passed for near functions and a far pointer for far functions, and it's impossible to determine the argument type until runtime; consequently, varargs is used to resolve the problem.

The far versions of _vector_new_ and _vector_vnew_ are used in the small data memory models for arrays of far classes, regardless of whether or not they're huge.

The far and near versions of _vector_vnew pass a second argument, zero, to the constructor. These versions are used for classes with virtual bases.

The following version of function _vector_delete_ is used only in the tiny, small, and medium memory models:

```
  void _vector_delete_
          (
          void near *  ptr,   // address of array
          size_t size,        // size of each object
          unsigned count,     // how many objects
          unsigned mode,      // how to call
          ...                 // destructor address passed here
          )
```

The following version of function _vector_delete_ exists in all memory models:

```
  void _vector_delete_
          (
          void far *  ptr,
          size_t size,
          unsigned long count,
          unsigned mode,
          ...
          )
```

The following list shows the interpretation of the mode for _vector_delete_:

far function	0x01
pascal call	0x02
far pointer	0x04
deallocate	0x08
stored element count	0x10
huge array (array > 64K)	0x40

The _vector_delete_ routines destroy arrays of class type. If mode has 0x08 set, the routines deallocate the space for the array after destroying the elements. When mode has 0x18 set, causing deallocation to occur and count to be used, the count is retrieved from the count field stored in a 16-bit word just below the array. Since the destructor for the class might be a near or a far function, and take a near or far this value, mode is passed to allow correct casting. A near pointer must be passed for near functions and a far pointer for far functions, and it's impossible to determine the argument type until runtime; consequently, varargs is used to resolve the problem.

The far version of _vector_delete is used in the small data memory models for arrays of far classes, regardless of whether or not they're huge.

Name mangling

There are four basic forms of encoded names in Borland C++:

1. @className@functionName$args

   This encoding denotes a member function functionName belonging to class className and having arguments args.

   Class names are encoded directly. The following example shows a className in an encoded name:

      @className@...

   The class name may be followed by a single digit; the digit value contains the following bits (these can be combined):

   0x01 the class uses a far vtable

   0x02 the class uses the -po calling convention

   0x04 the class has an RTTI-compatible virtual table;

this bit is only used when encoding the name of
the virtual table for the class

The digit is encoded as an ASCII representation of the bit mask value, with 1 subtracted (so that, for example, the class prefix for a class 'foo' that uses far vtables would be '@foo@0').

See the next section on the encoding of function names and argument types.

2. @functionName$args

This form of encoding denotes a function functionName with arguments args.

3. @className@dataMember

This form of encoding denotes a static data member dataMember belonging to
class className. Names of classes and data members are encoded directly. The
following example shows a member myMember in class myClass:

@myClass@myMember

4. @className@

This name denotes a virtual table for a class className. As mentioned
previously, class names are encoded directly.

Encoding of nested and template classes

The following form encodes a name of a class lexically nested within another
class:

@outer@inner@...

A template instance class encodes the name of the template class, along with the
actual template arguments, in the following way:

%templateName$arg1$arg2 ..... $argn%

Each actual argument starts with a letter, specifying the kind of argument it is:

t type argument

i nontype integral argument

g nontype nonmember pointer argument

m nontype member pointer argument

The first letter is followed by the encoded type of the argument. For a type argument, this code also represents the argument's actual value. For other kinds of arguments, the type code is followed by $ and the argument value, encoded as an ASCII number or symbol name. An instance of template<class T,int size> whose name is vector<long,100> is encoded as shown in the following example:

```
%vector$tl$ii$100%
```

Encoding of function names

The encoded functionName might denote either a function name, a function such as a function such as a constructor or destructor, an overloaded operator, or a type conversion.

Ordinary functions

Ordinary function names are encoded directly, as shown in the following examples:

```
foo(int) --> @foo$qi
sna::foo(void) --> @sna@foo$qv
```

The string $qi denotes the integer argument of function foo; '$qv' denotes no arguments in sna::foo.

The following information covers argument encoding in more detail. Constructors,
destructors, and overloaded operators are encoded with a $b character sequence,
followed by a character sequence from the following table:

Character Sequence	Meaning
ctr	constructor
dtr	destructor
add	+
adr	&
and	&
arow	->
arwm	->*
asg	=
call	()
cmp	~
coma	,
dec	--
dele	delete
div	/
eql	==
geq	>=
gtr	>
inc	++
ind	*
land	&&
lor	||
leq	<=
lsh	<<
lss	<
mod	%
mul	*
neq	!=
new	new
not	!
or	|
rand	&=
rdiv	/=
rlsh	<<=
rmin	-=
rmod	%=
rmul	*=
ror	|=
rplu	+=
rrsh	>>=
rsh	>>

rxor	^=
sub	-

subs	[]
xor	^

nwa	new []
dla	delete []

The following examples show how arguments are encoded with character sequences, add, ctr, and dtr from the previous table:

```
operator+(int)  -->     @$badd$qi
plot::plot()    -->     @plot@$bctr$qv
plot::~plot()   -->     @plot@$bdtr$qv
```

The string $qv denotes no arguments in the plot constructor or destructor.

Type conversions

Encoding of type conversions is accomplished with the $o character sequence, followed by the distinguishing return type of the conversion as part of the function name. The return type follows the rules for argument encoding, explained later. The lack of arguments in a conversion is made explicit in the mangling by adding $qv to the end of the encoded string.

Example:

```
foo::operator int()     -->     @foo@$oi$qv
foo::operator char *()  -->     @foo@$opzc$qv
```

The i following $o in the first example denotes int; the pzc in the second example denotes a near pointer to an unsigned char.

Encoding of arguments

The number and conbinations of function arguments make argument encoding the most complex aspect of name mangling.

Argument lists for functions begin with the characters $q. Type qualifiers are then encoded as shown in the following table:

Character Sequence	Meaning
up	huge
ur	_seg
u	unsigned
z	signed
x	const
w	volatile

Encoding of built-in types follows that for applicable type qualifiers, in accordance with the following table:

Character Sequence	Meaning

v	void
c	char
s	short
i	int
l	long
f	float
d	double
g	long double
e	...

Encoding of non-built-in types follows that for applicable type

qualifiers, in accordance with the following table:

Character Sequence	Meaning
<a digit>	(an enumeration or class name)
p	near *
r	near &
m	far &
n	far *
a	array
M	member pointer (followed by class and base type)

The appearance of one or more digits indicates that an enumeration or class name follows; the value of the digit(s) denotes the length of the name, as shown in the following examples:

```
foo::myfunc(myClass near&)      is mangled as @foo@myfunc$qr7myClass
foo::myfunc(anotherClass near&) is mangled as @foo@myfunc$qr12anotherClass
```

A character x or w may appear after p, r, m, or n to denote a constant or volatile type qualifier, respectively. The character q appearing after one of these characters denotes a function with arguments the follow in the encoded name, up to the appearance of a $ character, and finally a return type is encoded. The following example show how these encoding rules are applied:

```
@foo@myfunc$qpxzc      is mangled as   foo::myfunc(const char near*)
@func1$qxi             is mangled as   func1(const int)
@foo@myfunc$qpqii$i    is mangled as   foo:myfunc(int (near*)(int,int))
```

Array types are encoded as a, followed by a dimension encoded as an ASCII decimal number and a $, and finally the element type, as shown in the following example.

```
foo( int (*x)[20] )    is mangled as   @foo$qpa20$i
```

Encoded arguments are concatenated in the order of appearance in the function call. The character t followed by an ASCII character encodes the arguments when

a number of identical nonbuiltin types are function arguments. The ASCII

character, ranging from ASCII 31H - 39H and 61H - 7FH (1 to 9 and a onward), denotes which argument type to duplicate, as shown in the following example:

```
@plot@func1$qdddiiilllpzctata is unmangled to
        plot::func1(double, double, double, int, int, int, long, long, long,
        char near*, char near*, char near*)
```

The two duplicate ta character sequences at the end of the encoded name denote the tenth argument, encoded as pzc.

Dynamically dispatchable virtual tables

The DDVT table always precedes the 'regular' virtual table for the given class. The DDVT is located at negative offsets from the virtual table pointer. The following layout shows the format of the DDVT:

```
void (far *fpt[count])();
unsigned idt[count];
unsigned count;
void *basep;
the regular virtual table starts here:
void (*vtab[])();
```

The fpt and idt tables contain the addresses and IDs, respectively, of all DDVT functions introduced or overridden in the class. The count holds the number of entries in the tables. basep holds the address of the virtual table for the base class or zero if the class has no base; the size of the base class pointer is the same as the virtual table pointer for the class. The pointer is a far pointer for huge classes.

For example, consider the following two classes:

```
  struct  base
  {
    virtual f() = [11];
    virtual g() = [22];
    virtual h();
  };

  struct  der:base
  {
    f();
    virtual i() = [33];
    h();
  };
```

The following table is the DDVT/virtual table for class base:

```
                dd      @base@f$qv      ; addr of foo::f()
                dd      @base@g$qv      ; addr of foo::g()
```



```
                dw      11              ; ID for f()
                dw      22              ; ID for g()

                dw      2               ; 2 entries in DDVT

                dw      0               ; no base class
  base_vtable:
                dd      @base@h$qv      ; addr of base::h()
```

The following table is the DDVT/virtual table for class der:

```
                dd      @der@f$qv       ; addr of der::f()
                dd      @der@i$qv       ; addr of der::i()

                dw      11              ; ID for f()
                dw      33              ; ID for i()

                dw      2               ; 2 entries in DDVT

                dw      base_vtable     ; base class vtable addr
  der_vtable:
                dd      @der@h$qv       ; addr of  der::h()
```

CHAPTER

2

Object file contents

This chapter covers the comment records sent to

the object file by Borland C++ version 4.0. Other Borland compilers may not emit all of the records described here. The comment records are actually Intel Object Module Format (OMF-86) Comment records with the following specifications:

Value or Length	Description
0x88	COMMENT record byte
2 bytes	record length
0x00	A control byte (always zero)
1 byte	Comment record class (see below)
n bytes	Data (depends on Comment record class)
1 byte	Checksum

For fields described in this document, strings are stored as Pascal-style strings with a leading length byte, which might be zero. A zero length byte indicates a null string. An index is an OMF-86 index field. That is, if the value is below 128, then the index is a byte field with the index value; otherwise, the field is two bytes. The first byte has the high bit set and the remaining bits are the seven high-order bits of the index. The second byte is the low-order 8 bits of the index.

Type indices in the are the type indices defined for the .EXE file tables. Immediate indices 0 to 23 refer to scalar types. Type index 0 indicates an unknown type. Any type index higher than 23 indicates the index of a type record defined in the current file. Each type record contains its

own index, since the output of type records isn't necessarily in index order.

The official Intel-type index fields are always

zero, because MS-Link uses them for special purposes.

The order of comment records inside the object file is fairly flexible. Unless the description of a comment record specifies ordering requirements, the comment record might appear anywhere between the module header and module end records. The must appear immediately after the module header record and before any other type records. The compiler identification need not be for Turbo C++, nor is it absolutely necessary that a compiler id record appear at all.

Turbo object file comment records

This section dissects each comment record class. The memory location of the record class and its name appear on the left-hand side of the page, and a description of the record is located on the right-hand side of the page.

0x00 Compiler identification string

string
 A descriptive name reflecting the name of the translator used to generate this object file. For instance "Turbo Assembler Version 2.0".

0xe0 External symbol type index

index
 The type index of an external symbol. External symbols must be placed one symbol per EXTDEF record. This comment record supplies the type index of the external symbol located just previous to it in the object file.

If the debug information version record 0xf9 appears in the object file, the following fields are represented:

index
 The index of the source file that caused the record to be emitted.

word
The line number of the instruction in the source code line that caused the record to be emitted. The word is present only if the previous index is nonzero.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

0xe1 Public symbol type index
index
The type index of a public symbol. Public symbols must be placed one symbol per PUBDEF record. This comment record supplies the same type index as the public symbol located just previous to it in object file.

byte
If the symbol is a function with a valid BP, the byte contains the third bit set to one (hex 0x8), and the upper four bits set to the number of words between the BP value and the return address.

If the debug information version record (0xf9) appears in the object file, the following fields are present:

index
Index of source file that caused this record to be emitted.

word
The line number of the instruction in the source

code line that caused the record to be emitted.The word is present only if the previous index equals nonzero.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.

If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word, and this is a reference.

0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

0xe2 Structure member definition

Typically, all the members of a single structure are written to a single member record. If the number of members is so great that the OMF record exceeds 8K, or if the OMF record exceeds 8K for some other reason, the members of a single structure might be spanned across multiple records. Only the last member of the structure has the terminating bit (the high bit of the first byte) set. No more than one structure can appear in a member definition record.

If the debug version record 0xf9 doesn't appear in the object file, then one or more member definition records for a structure are written immediately before the type record for that structure; otherwise, the structure member

definition records must appear after the type for the structure, and after all the types that the member definition records reference.

* A consecutive set of member definition records. Each record consists of the following information:

1st byte:
  * 0x60 Static member

  * 0x50 Conversion

  * 0x48 Member function, which might be combined with the following bits:

    * 0x01
      destructor
    * 0x02
      constructor
    * 0x03
      static member function
    * 0x04
      virtual member function

If none of the previous values are present, then the following interpretation of the byte applies:

low six bits
  If the member is a bit field, this field represents the number of bits in the field; otherwise, the field is set to zero.

seventh bit
  This bit is set to zero if next bit is a normal member or to one if the next bit is a New Offset record.

high bit
  This bit is set to zero if there are more members in the current structure or to one if this is the last member in the structure.

For normal members the following rule applies:

string
  The member name. A zero byte is used for unnamed members. Since no explicit offset for each member is given, offsets are computed by counting the length of each member. When holes exist from bit fields not filling a byte or

when word alignment is used, an unnamed member is emitted. Such a member is always a bit field member with the appropriate number of pad bits. Although the compiler currently behaves according to this description, it accepts nonbit field unnamed members.

index
The member type. For conversions, this index specifies the target type of the conversion, for example int for "operator int();".

For New Offset members the following information applies:

double word
The new byte offset of the records that follow it. The double word allows variant records, since each variant portion can be started with a New Offset member. As a double word, this field is suitable for large structures.

0xe3 Type definition

One type is defined in each type definition record. The format of the type record depends on the type identification (TID) byte. See TID values defined in the EXE debug table format beginning on page 51.

TLINK defines a set of universal scalar types to save space in the object files. For integer range types, the type is stored with the maximum range for each type. If an index of less than twenty-four (decimal) appears in the object file, one of the pre-assigned types is indicated, and no type definition appears in the object file. The following list shows the set of and their assigned indices:

Index	Type
1	void
2	signed char
4	signed short int
6	signed long int
8	unsigned char
10	unsigned short int

12	unsigned long int
14	float
15	double
16	long double
17	Pascal 6-byte real

18	Pascal boolean
19	Pascal character type
21	8-byte signed range
22	8-byte unsigned range
23	10-byte value (tbyte)

index
The index of the type being defined. All types must have a valid index of twenty-four (decimal) or greater, and the indices must be unique within the object file. There's no requirement to write types in any particular order. All of the type indices for a given file form a contiguous block beginning at twenty-four and proceeding to the highest numbered index. Since some types occupy eight bytes and others sixteen bytes in the .EXE file, the TID values requiring sixteen bytes reserve their own type index as well as the next higher type index.

string
The type name, if any exists. For C, the type name is used only for structure, union, and enum tags. For Pascal, any type might be named.

word
The size in bytes of the type.

TID byte
This is the TID of the type being defined. These following list shows the :

Name	Value	Description
TID_VOID	0x00	Unknown.
TID_LSTR	0x01	Basic literal string.

TID_DSTR	0x02	Basic dynamic string.
TID_PSTR	0x03	Pascal style string.
TID_SCHAR	0x04	1 byte signed integer range.
TID_SINT	0x05	2 byte signed integer range.
TID_SLONG	0x06	4 byte signed integer range.
TID_SQUAD	0x07	8 byte signed integer.
TID_UCHAR	0x08	1 byte unsigned integer range.
TID_UINT	0x09	2 byte unsigned integer range.

TID_ULONG	0x0A	4 byte unsigned integer range.
TID_UQUAD	0x0B	8 byte unsigned integer.
TID_PCHAR	0x0C	Pascal character range (no arithmetic).
TID_FLOAT	0x0D	IEEE 32-bit real.
TID_TPREAL	0x0E	Turbo Pascal 6-byte real.
TID_DOUBLE	0x0F	IEEE 64-bit real.
TID_LDOUBLE	0x10	IEEE 80-bit real.
TID_BCD4	0x11	4 byte BCD.
TID_BCD8	0x12	8 byte BCD.
TID_BCD10	0x13	10 byte BCD.
TID_BCDCOB	0x14	COBOL BCD.
TID_NEAR	0x15	Near pointer.
TID_FAR	0x16	Far pointer.
TID_SEG	0x17	Segment pointer.
TID_NEAR386	0x18	386 32-bit offset pointer.
TID_FAR386	0x19	386 48-bit far pointer.
TID_CARRAY	0x1A	C array - 0 based.
TID_VLARRAY	0x1B	Very Large 0 based array.
TID_PARRAY	0x1C	Pascal array.
TID_ADESC	0x1D	Basic array descriptor.
TID_STRUCT	0x1E	Structure.
TID_UNION	0x1F	Union.
TID_VLSTRUCT	0x20	Very Large Structure.
TID_VLUNION	0x21	Very Large Union.
TID_ENUM	0x22	Enumerated range.
TID_FUNCTION	0x23	Function or procedure.
TID_LABEL	0x24	Goto label.
TID_SET	0x25	Pascal set.
TID_TFILE	0x26	Pascal text file.

TID_BFILE	0x27	Pascal binary file.
TID_BOOL	0x28	Pascal boolean.
TID_PENUM	0x29	Pascal enumerated range (no arithmetic).
TID_PWORD	0x2A	Pword
TID_TBYTE	0x2B	Tbyte
TID_SPECIALFUNC	0x2D	Member/Duplicate function
TID_CLASS	0x2E	C++ Class
TID_HANDLEPTR	0x30	Handle based ptr
TID_MEMBERPTR	0x33	Type pointed to by a class member pointer.
TID_NREF	0x34	Near reference
TID_FREF	0x35	Far reference
TID_NEWMEMPTR	0x38	New stype member ptr

The format of the remainder of the type record depends on the TID byte as shown in the following table:

Simple types

TID_VOID	TID_FLOAT	TID_BCD8	TID_TFILE
TID_LSTR	TID_TPREAL	TID_BCD10	TID_BOOL
TID_DSTR	TID_DOUBLE	TID_ADESC	TID_SCHAR
TID_SQUAD	TID_LDOUBLE	TID_STRUCT	TID_PWORD
TID_UQUAD	TID_BCD4	TID_UNION	TID_TBYTE

Pascal string type
TID_PSTR

byte	The maximum size of the string.

Labels
TID_LABEL

byte	Zero if near, one if far.

Integral range types

TID_SCHAR TID_SLONG TID_UINT TID_PCHAR
TID_SINT TID_UCHAR TID_ULONG

The integral range types are a hierarchy of related types that form a tree. The root of the tree is the general type, which is stored explicitly as a range. The parent type is zero and the lower and upper bounds are the entire range of values storable in the size of memory indicated by the TID. The bound values are interpreted as signed or unsigned according to the TID. The Pascal character TID (TID_PCHAR) is stored as an unsigned character-sized range, except arithmetic isn't allowed on objects of Pascal character type.

For all types, a 4-byte upper and lower bound value allows standard treatment of range checking.

The sub-fields are stored as shown in the following list:

* index The parent type index

* double word
  The lower bound of the range

* double word
  The upper bound of the range

Cobol-style BCD

TID_BCDCOB

byte The position of the decimal point. The number of total digits is determined from the size, using 2 digits per byte, except for the last byte, which has one digit and a sign. The decimal position is the number of digits to the right of the decimal point.

Pointer types

TID_NEAR TID_SEG TID_FAR386 TID_FREF
TID_FAR TID_NEAR386 TID_NREF

All pointer types have an index field for the pointed-to type. All have an additional byte field following the pointed-to type field that consists of extra information as follows:

TID_NEAR and TID_NEAR386

The segment base of the pointer:

Value	Segment register
0x0	segment register unspecified.
0x1	ES relative
0x2	CS relative
0x3	SS relative
0x4	DS relative
0x5	FS relative
0x6	GS relative

TID_FAR and TID_FAR386

0x0	far pointer arithmetic (no segment adjustments).
0x1	huge pointer arithmetic (segment adjustments to avoid offset wrap-around).

TID_SEG

0x0 ignored

TID_NREF

0x0 ignored

TID_FREF

0x0 ignored

Array types

TID_CARRAY

index	The index of the element type. The dimension of the array is determined by dividing the size of the overall array by the size of each element. No padding is assumed between array elements.

TID_VLARRAY

word	The upper 16 bits of the array size. This word is placed so that the normal type size field and this one can be considered a double word size.
index	The index of the element type. The dimension of the array is determined as it is for normal C arrays.

TID_PARRAY

index	The element type.
index	The type of the dimension. The number of elements in the array and their indices are determined by the dimension type, which is normally some sort of integral or enum range.

Very large structure types

TID_VLSTRUCT and TID_VLUNION

word — The upper 16 bits of the size of the struct or union. This word is placed so that the upper 16 bits and the normal type size can be considered a double word size.

Enumerated types

TID_ENUM and TID_PENUM

index — The index of the parent type.

word — The lower bound of the range (considered a signed integer range).

word — The upper bound of the range (considered a signed integer range).

If the debug information version record (0xf9) has appeared in the object file, then the following field is present:

index — Index to the first member of the enum. That is, this is an index to the structure member definition record that defines members to the enum.

Function types

TID_FUNCTION

index — The type index of the type returned.

byte — The language modifier byte:
0x0 Near C function
0x1 Near Pascal function
0x2 Unused.
0x3 Unused.
0x4 Far C function
0x5 Far Pascal function
0x6 Unused.
0x7 Interrupt function.

byte This byte is set to one if the function accepts a variable number of arguments; otherwise, it is zero.

Sets

TID_SET

index The parent type.

Binary files

TID_BFILE

index The element type.

Member/duplicate functions

TID_SPECIALFUNC

index The type index of the return type.

byte Language modifier byte (same as regular functions).

byte Bit 0 is set to indicate a member function;
bit 1 is set to indicate a duplicate function;
bit 2 set to indicate an operator function;
bit 3 set to indicate internal linkage;
bit 4 set to indicate this is a Pascal function passing
'this' as last parameter.

index The type index of the class if the function is a member function.

index Word offset in the virtual table if the function is a member function.

name if the function is a nonlocal member function.

this should appear as a local symbol in the second inner scope of a member function, not in the outermost (parameter) scope.

C++ Class
TID_CLASS

index The class index for this class.

Pointed-to members

TID_MEMBERPTR

index The type index of the pointed-to type.

index The class index of the class whose members are pointed to.

New style pointed-to members
TID_NEWMEMBERPTR

byte Member pointer flags.

index The type index of the pointed-to type.

index The class index of the class whose members are pointed to.

0xe4 Enum member definitiona
Typically, all members of a single enum are written to a single member record. If the number of members is so great that the OMF record exceeds 8K, or if the OMF record exceeds 8K for some other reason, the members of a single enum might be spanned across multiple records. Only the last member of the enum has the terminating bit (high bit of first byte) set.

No more than one enum can appear in a member definition record.

If the debug version record 0xf9 has not appeared in the object file, then one or more member definition records for an enum are written immediately before the type record for that enum; otherwise, the enum member definition record must appear after the type for the enum.

Each record in a consecutive set of member definition records consists of the following data:

byte	0x80 for the last member of the enum, otherwise this byte is set to zero.
string	The member name.
word	The member value.

0xe5 Begin scope record

Scopes are defined by a pair of begin-scope end-scope records. The relationships of nested scopes are specified by enclosing the begin/end records of one scope between the begin/end records of another. Local symbols are defined for a scope by having the locals definition records between the begin/end records of the scopes.

index	The segment index of the segment containing the scope. This segment must be the same as the segment of the starting address.
word	The offset, relative to the code segment, of the start of this scope.

0xe6 Locals definition record

This record consists of a set of symbol definitions,all local to the innermost enclosing scope.

The following list shows the contents for each symbol:

string The symbol name.

index The symbol type index.

byte The symbol class byte.

The remainder of the symbol depends on the value of the symbol class byte:

SC_TYPEDEF (6) and
SC_TAG (7)

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_STATIC (0)
index The group index of the symbol.

* index The segment index of the segment

containing the symbol. For an absolute symbol this must be an absolute segment.

* word The offset relative to the given segment of the symbol.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.

If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word, and this is a reference.

0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_ABSOLUTE (1)

index The segment index of the segment containing the symbol. For an absolute symbol the index must be an absolute segment.

* word The offset relative to the given segment of the symbol.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.

If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word,

and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_AUTO (2) and
SC_PASVAR (3)
word The signed offset, relative to BP, of the symbol. For Pascal variable parameter symbols, the location contains the address of the symbol.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_REGISTER (4)
byte A register id. Register ids map to registers as follows:

0x00 AX	0x01 CX	0x02 DX	0x03 BX
0x04 SP	0x05 BP	0x06 SI	0x07 DI
0x08 AL	0x09 CL	0x0A DL	0x0B BL
0x0C AH	0x0D CH	0x0E DH	0x0F BH
0x10 ES	0x11 CS	0x12 SS	0x13 DS
0x14 FS	0x15 GS	0x18 EAX	0x19 ECX
0x1A EDX	0x1B EBX	0x1C ESP	0x1D EBP

0x1E ESI 0x1F EDI

If the register ID value is greater than 0x28,
the field then specifies an offset (minus 0x28)
into the optimized symbols table which is the
live range information for this variable.

If the debug information version record 3.1 appears in the object file,
the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.
If this is non-zero, there follow a set of line
numbers with special encodings.

Each line number is stored as a delta from the previous line number,
with the starting line number after a file index being 0. By
default, the delta is stored in a byte, with the low 6 bits being
the line number, and the 7th bit being a toggle to specify whether
this is a reference or an assignment. If the 7th bit is set, it is
an assignment. If the byte is greater then or equal to 0xf0, then
it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute
word line number following.

0xfe The absolute line number is stored in the next word,
and this is a reference.

0xfd The absolute line number is stored in the next word,
and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_CONST (5)

dword The 32-bit constant value.

If the debug information version record 3.1 appears in the object file,
the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.
If this is non-zero, there follow a set of line
numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word, and this is a reference.

0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_OPT (8)

index The number of entries for this local. Each entry represents a different location for the local for a different set of code offsets; hence, a single SC_OPT sub-record represents a complete list of optimized symbol records for the debugger. The following section describes the format of the entries:

* word Starting offset of the live range of the variable. The offset is relative to the offset of the outermost enclosing scope.

* word Ending offset of the live range of the variable. The offset is relative to the offset of the outermost enclosing scope.

* byte One of SC_AUTO, SC_PASVAR or SC_REGISTER.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.

If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word,

and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_AUTO and SC_PASVAR

* word The signed offset, relative BP, of the symbol. For Pascal variable parameter symbols, the location contains the address of the symbol.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

SC_REGISTER

* byte The register id.

SC_OPT is complex to be able to handle the difficulties encountered when a variable lives in a register, is spilled to the stack, and then is moved to a register again. This complexity does not exist in Borland C++ Version 4.0, because split live ranges are not implemented; however this specification was written with the intent of covering all contingencies, such as the compiler getting smarter with live ranges.

If the debug information version record 0xf9 appears in the object file, the following fields are present:

* index Index of source file that defined this record to be emitted.

* word Line number of source code that caused this record to be emitted. This word is present only if the previous index is nonzero.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.
0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

0xe7 End of scope
word The offset relative to the code segment of the end of the scope.

0xe8 Select source file

This comment is placed before any line numbers for a particular file. It's not needed if line numbers aren't generated before the next source file is encountered.

index	The source-file index of the new source file. If no further data exists in this record, then this index refers to an existing source file specified in a Select Source File record; otherwise, it is followed by the source file name and time stamp.
string	The source file name, relative to the current path.
dword	The DOS date and time stamp for the file.

0xe9 Dependency file definition

This comment is included for each distinct source and include file in the object module. The records should be placed near the top of the object file, since a MAKE utility must scan the file for dependency records.

The first dependency record must precede any noncomment record other than the THEADR record.

dword	The DOS date and time stamp for the file.
string	The name of the source file. The string opens the file. For Turbo C, if an found in a -I directory, the directory name is prepended to the filename,

allowing the MAKE utility to check dependencies by simply retrieving the file time stamp without searching through a path. If the record has zero length, then there are no more dependency records in the object file.

0xea Compile parameters record

1st byte The source language for this object file. If an assembler source contains debugging information, the language is the one specified in the source, not assembly language. The following language types are defined:

- 0 - unspecified
- 1 - C
- 2 - Pascal
- 3 - Basic
- 4 - Assembly
- 5 - C++

2nd byte

1 bit This bit is one if underbars were prepended to C language source symbols, otherwise, it's zero.

3 bits These bits specify the and, therefore, the default pointer sizes for this source:

- 0 - Tiny
- 1 - Small
- 2 - Medium
- 3 - Compact
- 4 - Large
- 5 - Huge
- 6 - 80386 Small
- 7 - 80386 Medium
- 8 - 80386 Compact
- 9 - 80386 Large

Code pointers are near in the Tiny, Small, and Compact models, and far otherwise. Data pointers are near in the Tiny, Small and Medium Models, and far otherwise.

The 80386 models are analogous to the corresponding 8086 models: A near has a 32-bit offset, and a far 80386 pointer is a 48-bit pointer.

0xeb External symbol matched type index

The following fields are repeated as many times as necessary to fit in the record.

* string — The symbol name itself.
* index — The type index of the symbol.

If the debug information version record 0xf9 appears in the object file, the following fields are present:

* index — Index of the source file that caused this record to be emitted.
* word — Line number of source code line that caused the record to be emitted. This word is present only if the previous index is nonzero.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.

If this is zero, there is no reference info.

If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.

0xfe The absolute line number is stored in the next word, and this is a reference.

0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

0xec Public symbol matched type index

The following fields are repeated as many times as necessary to fit in the record.

* string — The name of the public symbol.

* index	The type index for the symbol.
* byte	This byte contains the same information as the valid BP byte previously defined.

If the debug information version record 0xf9 appears in the object file, the following fields are present:

* index	Index of source file that caused this record to be emitted.
* word	Line number of source code line that caused this record to be emitted. This word is present only if the previous index is nonzero.

If the debug information version record 3.1 appears in the object file, the following fields are present:

First is a 16-bit file index.
If this is zero, there is no reference info.
If this is non-zero, there follow a set of line numbers with special encodings.

Each line number is stored as a delta from the previous line number, with the starting line number after a file index being 0. By default, the delta is stored in a byte, with the low 6 bits being the line number, and the 7th bit being a toggle to specify whether this is a reference or an assignment. If the 7th bit is set, it is an assignment. If the byte is greater then or equal to 0xf0, then it is a special encoding with the following meaning:

0xff Next byte/word is a file index, with an absolute word line number following.
0xfe The absolute line number is stored in the next word, and this is a reference.

0xfd The absolute line number is stored in the next word, and this is an assignment.

The remaining values are reserved.

A reference info object is terminated by 2 zero bytes.

0xed Class definition

This record describes classes.

The class definition records have the following format:

* byte 0 = class description

Class descriptions

Class descriptions have the following format:

* index Class index for the class. TID_CLASS and TID_MEMBERPTR type records refers to this index.

* word Offset (in bytes) of the .

If the debug information version record 0xf9 appears in the object file, the following field is present:

* index Index to the first member of the structure; that is, an index to the structure member definition record that defines members to this class.

byte Info bits:
bit 0: Class declared as 'struct'
bit 1: 'huge' class (far vtable pointer)
bit 2: 'far' class (far 'this' pointer)
bit 3: 'far' class that uses 'near'
vbase pointers
bit 4: a union

* index The number of parent indices that follow.

* word(s) Indices of parent classes (repeated); the highest bit is set for virtual base classes.

Note

If a class definition appears between begin-scope and end-scope records, it is interpreted as a locally defined class.

0xee Coverage offset record

To aid in profiling, the compiler emits offsets to delimit the start and end of basic blocks. The offsets, if taken pairwise, define the beginning and end of a basic block. The offsets are relative to the specified logical segment defined in the object file.

* index	The segment index of the segment, corresponding to the offsets that follow.
* array of words	Each word corresponds to an offset. The length of the array is dictated by the length of the OMF record.

0xf5 Begin large scope record

Scopes are defined by a pair of begin-scope, end-scope records. The relationships of nested scopes are specified by enclosing the begin/end records of one scope between the begin/end records of another. Local symbols are defined for a scope by having the locals definition records between the begin/end records of the scopes.

* index	The segment index of the segment containing the scope. This segment must be the same as the segment of the starting address.

* double word — The large offset, relative to the code segment, of the start of this scope.

0xf6 Large offset locals definition record

This record consists of a set of symbol definitions, all local to the innermost enclosing scope.

The following list shows the contents for each symbol:

* string — The symbol name.
* index — The symbol type index.
* byte — The symbol class byte.

The remainder of the symbol depends on the value of the symbol class byte:

SC_STATIC (0)

index The group index of the segment containing the symbol.
* index — The segment index of the segment containing the symbol.
* double word — The large offset relative to the given segment of the symbol.

SC_ABSOLUTE (1)
index The segment index of the segment containing the symbol. For an absolute symbol the segment must be absolute.

* double word The large offset relative to the given segment of the symbol.

SC_AUTO (2) and
SC_PASVAR (3)
double word The signed large offset, relative to BP, of the symbol. For Pascal variable parameter symbols, the location contains the address of the symbol.

0xf7 Large end of scope
double word The large offset relative to the code segment of the end of the scope.

0xf8 Member function
This record has to be located immediately after the outermost begincope record for every member function. It contains one field:

* string Mangled name of member function.

0xf9 Debug Information Version

This record immediately follows the compiler identification comment record. It specifies the major and minor version numbers of the debug information present in this file. If the major version of the debug information is higher than the major version that the linker understands, then all debug information is ignored. The minor version is ignored by the linker and is only used for diagnostic tools such as TDUMP. Borland C++ Version 4.0 emits version 4.01 debugging information. The record contains two fields:

* Major byte	Major version of the debug information.
* Minor byte	Minor version of the debug information.

0xfa Module optimization flags

Thi record presents the module optimization flags previously described. The compiler is responsible for emitting flags, which the linker passes unchanged to the debug information in the .EXE file.

* dword	Optimization flags.

The following flags are currently defined:

```
#define MO_globalCSEs        0x0001
#define MO_localCSEs         0x0002
#define MO_inductVars        0x0004
#define MO_codeMotion        0x0008
#define MO_regAlloc          0x0010
#define MO_loadOptim         0x0020
#define MO_loopOpt           0x0040
#define MO_intrinsics        0x0080
#define MO_deadStorElim      0x0100
#define MO_copyProp          0x0200
#define MO_jumpOpt           0x0400
```

```
  #define MO_speed_size        0x0800
  #define MO_noAliasing        0x1000
```

.OBJ extensions for 32 bits

The .OBJ spec was originally designed for the 16-bit world. Fortunately, its designers only allotted even numbered record types to the standard 16-bit records. The following extension uses the odd numbered record types to represent the 32-bit equivalents where needed.

SEGD32 (99h)	Size field is 32 bits.
LEDA32 (A1h)	Offset field is 32 bits.
LIDA32 (A3h)	Offset field is 32 bits, iteration count fields are 32 bits.
PUBD32 (91h)	Offset field is 32 bits.
MODE32 (8Bh)	Starting offset field is 32 bits.
LINN32 (95h)	Offset is 32 bits.
FIXU32 (9Dh)	Offset and displacement are 32 bits.

In the SEGDEF and SEGD32 records, the ACBP byte is redefined as follows:

Bit 0 (formerly InPage) now means USE32 when set.

The align types are extended to include DWORD alignment after PAGE alignment.

This specification can be extended to include other record types, as needed.

The 16-bit equivalent of any record can be used until one or more fields exceed the 16-bit size limitation. TASM uses such a minimalist approach in generating records to save space.

VIRDEF Records

The following modified record is provided for the linker to support unique instantiation of virtual tables, "out of line inlines" and various thunks the compiler generates. The mechanism is called

"" for and it is similar to an initializable COMDEF.

It begins with a change to the . A is identical to a COMDEF record with the exception that

the "segment type" must be a number in the range 1..0x5F (instead of the 0x61 and 0x62 far and near COMDEF types);

it is to be interpreted as a segment index, and may refer to any SegDef in the current module, with the meaning that the VIRDEF is to be appended to that segment IF it is instantiated;

the record format is like that for a near COMDEF, with a single length count.

The VIRDEF defines both a Public name and an External Index in the same way as a COMDEF does.

VIRDEFs cannot be resolved onto a Public or a COMDEF of the same name: any attempt to mix will be a link time error.

All VIRDEFs of the same name will be taken to be identical. When all sources files have been read and the linker has decided which modules are to be kept and which modules are to be discarded it scans the list of instances of each VIRDEF. It ignores instances which are in discarded modules, and selects the instance which is the first of the largest instances (or the first if all are equal in size). That instance is updated as the actual public symbol. Its segment is chosen (in the case where the VIRDEFs do not all attach to the same segment) and its module is noted. Only the LEDATA records from that module will be used, the others will be ignored.

VIRDEFs may be attached to either data or code segments. If a uniform choice of segment is not made and the code generated to reference the VIRDEF cannot reach the target then it generates fixup overflows in the usual way: it is not an error to have a single name of VIRDEF with

different segments unless it results in overflows.

A COMDEF may be seen as a "special case" of a VIRDEF, one which is attached to either BSS or an invented FAR segment, and which is never initialized with LEDATA.

When a reference is made to a VIRDEF from other object file records, the index that refers to the VIRDEF will be greater than 0x4000. To use the index, subtract 0x4000, and use it as a normal index.

These changes will not be compatible with Microsoft's LINK but only occur in C++ code.

CHAPTER

3

Symbol table format

TLINK's debugging output is written at the end of the load image in the .EXE file. An image that does not include extra information beyond the image size has no debug information. If extra data is written beyond the load image, check the first word for the number 0x52fb.

The debug information begins with a header describing the sizes of the remaining tables. This header is defined as follows:

```
  struct  debug_header
  {
    unsigned short  magic_number;   /* To be sure
  who we are */
    unsigned short  version_id;     /* In case we
  change things */
    unsigned long   names;          /* Names pool
  size in bytes */
    unsigned long   names_count;    /* Number of
  names in pool */
    unsigned long   types_count;    /* Number of
  type entries */
    unsigned long   members_count;  /* Structure
  members table */
    unsigned long   symbols_count;  /* Number of
  symbols */
    unsigned long   globals_count;  /* Number of
  global symbols */
    unsigned long   modules_count;  /* Number of
  modules (units)*/
    unsigned long   locals_count;   /* optional;
  can be filler*/
    unsigned long   scopes_count;   /* Number of
  scopes in table*/
```

```
  unsigned long   lines_count;    /* Number of
line nos */
  unsigned long   source_count;   /* Number of
include files */
  unsigned long   segment_count;  /* number of
segment records*/
  unsigned long   correlation_count;/* number
of segment/file */
                                  /*
correlations */
  unsigned long   image_size;     /* The number
of bytes in */
                                  /* the .EXE
file if the */
                                  /*
uninitialized part of */
                                  /* the data,
plus this */
                                  /* debug info
were removed. */
  void    far *debugger_hook;     /* A far ptr
into debugged */
                                  /* program,
meaning depends */
                                  /* on program
flags. For pascal */
                                  /* overlays,
is ptr to start of */
                                  /* data area
that contains info */
                                  /* contains
about the overlays. */
  unsigned char   program_flags;  /* A byte of
flags */
                                  /* 0x01 =
Case sensitive link */
                                  /* 0x00 =
Case insensitive link */
                                  /* 0x02 =
pascal overlay program*/
  unsigned stringsegoffset;       /* No longer
used */
  unsigned short data_count;      /* size in
bytes of data pool   */
```

```
  unsigned char filler;           /* to force
alignment */
  unsigned short extension_size;  /* 0, or 16,
for now */
};

struct header_extension
{
```



```
  unsigned long class_entries;         /*
number of classes */
  unsigned long parent_entries;        /*
number of parents */
  unsigned long global_classes;        /*
number of global classes */
                                       /* - NOT
USED */
  unsigned long scope_class_entries;   /*
number of scope classes */
  unsigned long module_class_entries;  /*
number of module classes /
  unsigned long CoverageOffsetCount;   /*
number of coverage offsets*/
  unsigned long NamePoolOffset;        /*
offset to start of name *
                                       /* pool.
This is relative */
                                       /* to
the symbols base */
  unsigned long BrowserEntries;     /* number
of browser info recs */
  unsigned long OptSymEntries;      /* number
of opt symbol recs */
  unsigned int DebugFlags;          /* various
flags  */
  unsigned long refInfoSize;        /* size in
bytes of ref */
                                    /* info
section */
  char    filler [14];              /* padding
*/
};

typedef struct      /* Trailer at end of NEW
EXE with debug info */
{
```

```
    unsigned short Signature;       /* 'NB'
  */
    unsigned short Version;         /* MS debug
  info version number */
    unsigned long Size;             /* Codeview
  header offset =     */
                                    /* (EOF -
  Size)                 */
  } TMSDbgTrailer;
```

The layout appears in the .EXE files as follows:

EXE header
fixups
EXE image

debug header
Symbol Table
Module Table
Source File Table
Scopes Table
Line Number Table
Segments Table
Correlation Table
Type Table
Members Table
Class Table
Parent Table
Scope Class Table
Module Class Table
Coverage Map Table
Coverage Offsets Table
Browser Definitions Table
Optimized Symbols Table
Module Optimization Flags Table
Reference Information Table
Names Table

For new .EXE files, there will be an 8-byte Codeview header immediately before the debug header, and an 8-byte Codeview trailer immediately after the names table. TD symbols tables can be told apart from Microsoft-generated tables by the value 0xFFFFFFFF in the last 4 bytes of the Codeview header.

All symbols, global or not, appear in the symbols area. The globals appear first, with module and local symbols following. The globals field specifies how many of the symbols are globals.

Identifiers are stored as indexes into the names pool. The index is to the relative identifier number (starting at 1). This way 64K distinct identifiers of arbitrary length can be stored. Names are stored uniquely, so that comparing indexes is as good as comparing strings. An identifier is stored in the pool as an ASCIIZ string (null-terminated string).

Symbols

```
struct symbol_record
{
  unsigned long   symbol_name;
  unsigned long   symbol_type;
  unsigned short  symbol_offset;
```



```
  unsigned short  symbol_segment;
  unsigned short  symbol_class : 3;
  unsigned short  has_valid_BP : 1;
  unsigned short  return_address_word_offset :
3;
};
```

The symbol table consists of a series of symbol definitions, sorted into ascending address order, with constant symbols (symbol_class == 5) at the end of each section (global or module local).

Note also that globals are all static, absolute, or typedefs.

No register globals are generated by Borland compilers at this time.

symbol_name is the index of the symbol name.

symbol_type is the index of the symbol type.

symbol_offset is interpreted according to the symbol_class field.

symbol_segment is the segment part of the symbol address for static symbols.

For new .EXE files, the top two bits of symbol_segment are used to provide information about symbols in DLLs as follows: If SR_SS_DllEntry bit is non-zero, then SR_SS_OrdinalFlag determines whether or not the SR_SS_Ordinal field of symbol_segment is an ordinal value or not.

For DLLs, symbol_offset is the name index of the module and symbol_name is name index of the DLL's entry point.

symbol_class is one of the following:

Value	Symbol class
0x0	Static, offset and segment give the address.
0x1	Absolute symbol. The segment and offset is the absolute address of the symbol.
0x2	Auto, offset is treated as signed, relative to BP.

0x3	Pascal var parameter. The offset is BP relative and is the location of the far pointer to the parameter.						
0x4	Register. Offset is a register ID as follows:						
0x00	AX	0x0A	DL	0x14	FS	0x20	ST(0)
0x01	CX	0x0B	BL	0x15	GS	0x21	ST(1)
0x02	DX	0x0C	AH	0x18	EA	0x22	ST(2)X
0x03	BX	0x0D	CH	0x19	EC	0x23	ST(3)X

0x04	SP	0x0E	DH	0x1A	ED	0x24	ST(4)X
0x05	BP	0x0F	BH	0x1B	EB	0x25	ST(5)X
0x06	SI	0x10	ES	0x1C	ES	0x26	ST(6)P
0x07	DI	0x11	CS	0x1D	EB	0x27	ST(7)P
0x08	AL	0x12	SS	0x1E	ESI		
0x09	CL	0x13	DS	0x1F	EDI		

0x5 Constant. Up to 4-byte constant stored in offset/segment.

0x6 Typedef. The offset field is ignored.

0x7 Structure/Union/Enum Tag. The offset is a type index.

```
#define SC_STATIC   0x0
#define SC_ABSOLUTE 0x1
#define SC_AUTO     0x2
#define SC_PASVAR   0x3
#define SC_REGISTER 0x4

#define SC_CONST    0x5
#define SC_TYPEDEF  0x6
#define SC_TAG      0x7

#define SR_SS_DllEntry     0x8000  /* symbol is
a dll entry   */
#define SR_SS_OrdinalFlag  0x4000  /* segment
is ordinal value */
#define SR_SS_Ordinal      0x3fff  /* mask to
obtain ordinal value */
```


The has_valid_BP field is defined for functions only. If the bit is zero, the function does not set up a BP stack frame, if the value is one then a valid BP is set up.

The return_address_word_offset field contains the

offset in words from BP where the return address can be found if the has_valid_BP field is not zero. The size of the return address is determined from the function type.

Modules

A module (or unit) consists of a set of objects, source files, and correlation records.

```
struct  module_header
{
  unsigned long   module_name;
  unsigned char   language;
  unsigned short  memory_model : 3;
  unsigned short  underbars_on : 1;
  unsigned long   symbols_index;
  unsigned short  symbols_count;
  unsigned short  source_files_index;
  unsigned short  source_files_count;
  unsigned short  correlation_index;
  unsigned short  correlation_count;
};

#define MM_TINY         0x0
#define MM_SMALL        0x1
#define MM_MEDIUM       0x2
#define MM_COMPACT      0x3
#define MM_LARGE        0x4
#define MM_HUGE         0x5
#define MM_SMALL386     0x6
#define MM_MEDIUM386    0x7
#define MM_COMPACT386   0x8
#define MM_LARGE386     0x9
```

module_name is the index of the module's name. This name is the source file name given to the compiler, including the extension.

symbols_index is the index of the first symbol in the symbol table for the module.

symbols_count is the number of symbols defined local to the module.

source_files_index is the index of the first source file record for the module.

source_files_count is the number of source files in the module.

correlation_index is the index of the correlation record for the module.

correlation_count is the number of correlation entries in the module.

language indicates the source language for the module.

Value	Language
0	Unknown
1	C
2	Pascal
3	Basic (not used)
4	assembly language
5	C++

memory_model determines default pointer sizes in type conversions.

underbars_on is non-zero if underbars should be prepended for cdecl-style symbols in any search context in this module.

Source files

```
struct  source_file
{
  unsigned long   source_file_name;
  unsigned long   time_stamp;
};
```

Each source file with line numbers in the executable code will have a source file record in the list module source files. There will always be at least one source file record per module (assuming there is any executable code in the module). Each include file containing code will generate a single source-file record per inclusion.

The line numbers for a segment within a source file will appear as a block in the line number table.

The source files in a module will appear in the order of their appearance in the compilation process. Thus the main source file appears first, followed by each of the include files. Note that if an include file doesn't have executable code (and therefore no source line numbers), it shouldn't be included here. Thus, for most source files with no code in include files, there will be only one file entry per module. Of course, if no executable code appears in a module, there is no need for a source file record.

The source file name will include any subdirectory information. Thus, if Turbo Debugger is run in the source directory (or with the source directory given in the appropriate TD option), it should be able to find all the source, even if it originated from some other source or had some peculiar file-name extension. For include files, the actual path name used to open the file is used. This way the debugger doesn't duplicate the compiler's include directory search logic.

The date/time stamp determines if the source file has changed since the time of the link.

Line numbers

```
struct  line_number
{
  unsigned short  line_number_value;
  unsigned short  line_number_offset;
};
```

line_number_value is the module line number.

line_number_offset is the offset of the line number relative to the segment value stored in the segment record referred to in the active correlation record.

Only unique offsets have line numbers stored. When a statement spans several lines, there can

be two line records with the same offset, but different line numbers.

The line number records are address sorted; they are not necessarily line-number ordered.

Scopes

```
struct  scope
{
  unsigned long   autos_index;
  unsigned short  autos_count;
  unsigned short  parent_scope;
  unsigned long   function_symbol;
  unsigned short  scope_offset;
  unsigned short  scope_length;
};
```

autos_index and autos_count define the symbol table area containing this scope's symbols. The auto_start is the index into the symbols table of the first variable local to the scope.

parent_scope is the index of the scope within the current module of the immediate enclosing scope.

scope_offset and scope_length defines the ranges of code addresses the scope is valid for. The segment is that stored in the segment record referred to in the active correlation record.

To handle nested units in pascal, there is a set of scopes at the beginning of the scopes table with a function_symbol of 0xffff. There is a one-to-one correspondence between these and the module (unit) records. These are the "unit scopes." The symbols that the record points to are the interfaced symbols of the unit.

The "uses scope" record has a function_parent of 0xfffe to establish the correct linking between the unit scope records. It does not contain information about the scope's symbols. Instead, autos_index is an index to the unit scope record

that refers to the interfaced symbols. To look up a name, the scopes are traced using the scope_parent records, but the symbols are accessed by referring to the corresponding unit scope record.

Segments

```
  typedef struct /* segment info */
```



```
  {
    unsigned short  mod_index;
    unsigned short  code_segment;
    unsigned short  code_offset;
    unsigned short  code_length;
    unsigned short  scopes_index;
    unsigned short  scopes_count;
    unsigned short  correlation_index;
    unsigned short  correlation_count;
  } segrec;
```

A segment record gives a code segment, offset, and length, and relates it to a particular module. It also gives an index into the scopes table for the scopes defined in the segment. The correlation table index and count allow the segment to be related to one or more source files and possibly to non-continuous groups of lines inside the files.

The segment records are address-ordered by segment and then by offset within the segment.

mod_index is the index of the module record for the corresponding module.

code_segment is the base address of the segment in the image.

code_offset is the offset from the base address of the segment in the image.

code_length is the length of the segment.

scopes_index is the index of the scope record of

the starting scope for this segment.

scopes_count is the count of scopes for this segment.

correlation_index is the index of the correlation record for the starting correlation for this segment.

correlation_count is the number of correlation records for this segment.

Segment/source file correlations

These records link a range of line numbers in a file to a particular segment record.


```
typedef struct
{
  unsigned short segment_index;
  unsigned short file_index;
  unsigned long  lines_index;
  unsigned short lines_count;
} correlation;
```

segment_index is the index of the segment record for this correlation.

file_index is the index of the source file record for this correlation.

lines_index is the index of the first line number record for this correlation.

lines_count is the number of line number records for this correlation.

Types

The type table consists of a set of 12-byte entries. Each type contains one or (for a few types) two entries.

The index value is used when a type is referred

to. Since no operations need to search the type table itself (all accesses will use index numbers), any type that occupies more than one entry will not have a type id byte for the upper half. Thus type records are effectively either 8- or 16-bytes long, depending on the particular type. Also, since only two sizes are present, a program can treat the table as effectively as a table of fixed size objects.

Simple types and common fields

The fields in the following table are common to all types.

Field	Size	Offset
type_id	1	0
type_name	4	1
type_size	2	5

type_name is 0 if the type is unnamed or is the name index of the type name.

type_size is the size in bytes of the object. This field is present in all type records.

type_id values are

```
#define TID_VOID        0x00        /* Unknown
or no type   */
#define TID_LSTR        0x01        /* Basic
Literal string */
#define TID_DSTR        0x02        /* Basic
Dynamic string */
#define TID_PSTR        0x03        /* Pascal
style string  */
```

Pascal strings
(12 bytes)

Field	Size	Offset
max_size	1	7

```
#define TID_SCHAR       0x04        /* 1 byte
signed range   */
#define TID_SINT        0x05        /* 2 byte
signed range   */
#define TID_SLONG       0x06        /* 4 byte
signed range   */
#define TID_SQUAD       0x07        /* 8 byte
signed int     */

#define TID_UCHAR       0x08        /* 1 byte
unsigned range */
#define TID_UINT        0x09        /* 2 byte
unsigned range */
#define TID_ULONG       0x0A        /* 4 byte
unsigned range */
#define TID_UQUAD       0x0B        /* 8 byte
unsigned int   */

#define TID_PCHAR       0x0C        /* Pascal
character type */
```

Ranges (24 bytes)

Field	Size	Offset
parent type	2	8
lower bound	4	12
upper bound	4	16

```
#define TID_FLOAT       0x0D        /* IEEE
32-bit real     */
#define TID_TPREAL      0x0E        /* Turbo
Pascal 6-byte real */
```



```
#define TID_DOUBLE      0x0F        /* IEEE
64-bit real     */
#define TID_LDOUBLE     0x10        /* IEEE
80-bit real     */
```

```
     #define TID_BCD4        0x11        /* 4 byte
     BCD           */
     #define TID_BCD8        0x12        /* 8 byte
     BCD           */
     #define TID_BCD10       0x13        /* 10 byte
     BCD          */
```

BCD COBOL (12 bytes)

Field	Size	Offset
decimal point	1	5

```
     #define TID_BCDCOB      0x14        /* COBOL
     BCD           */
```

Pointers (12 bytes)

Field	Size	Offset
extra info	1	7
pointed-to type	4	8

```
     #define TID_NEAR        0x15        /* Near
     pointer         */
     #define TID_FAR         0x16        /* Far
     pointer          */
     #define TID_SEG         0x17        /* Segment
     pointer      */
     #define TID_NEAR386     0x18        /* 386
     32-bit offset ptr*/
     #define TID_FAR386      0x19        /* 386
     48-bit far ptr   */
```

C arrays (12 bytes)

Field	Size	Offset
element type	4	8

```
     #define TID_CARRAY      0x1A        /* C array
     - 0 based    */
```

Very large arrays
(12 bytes)

Field	Size	Offset
object size	2	7
element type	4	9

```
#define TID_VLARRAY     0x1B        /* Very
Large 0 based array */
```

Pascal arrays
(24 bytes)

Field	Size	Offset
element type	4	8
dimension type	4	12

```
#define TID_PARRAY      0x1C        /* Pascal
array         */
```

Structs and unions
(12 bytes)

Field	Size	Offset
members index	4	8

```
#define TID_ADESC       0x1D        /* Basic
array descriptor */
#define TID_STRUCT      0x1E        /*
Structure           */
#define TID_UNION       0x1F        /* Union
*/
```

Very large structs
and unions
(24 bytes)

Field	Size	Offset
object size	2	7

members index 4 9

#define TID_VLSTRUCT 0x20 /* Very Large Structure */
#define TID_VLUNION 0x21 /* Very Large Union */

Enums (24 bytes)

Field	Size	Offset
lower bound	2	12
upper bound	2	14
members index	4	16

#define TID_ENUM 0x22 /* Enumerated range */

Functions (12 bytes)

Field	Size	Offset
language	0:7	7:0 *
accepts var. args.	0:1	7:7
return type	4	8

* These should be read as byte:bit

#define TID_FUNCTION 0x23 /* Function or procedure*/

Labels (12 bytes)

Field	Size	Offset
near/far	1	7

```
#define TID_LABEL   0x24            /* Goto
label           */
```

Sets (12 bytes)

Field	Size	Offset
parent type	4	8

```
#define TID_SET     0x25        /* Pascal set
*/
```

Binary files
(12 bytes)

Field	Size	Offset
element type	4	8


```
#define TID_TFILE       0x26        /* Pascal
text file     */
#define TID_BFILE       0x27        /* Pascal
binary file   */
```

Function prototypes
(24 bytes)

Field	Size	Offset

language	0:7	7:0 *
accepts var. args.	0:1	7:7
return type	4	8
parameter start	2	12

* These should be read as byte:bit

```
#define TID_BOOL        0x28        /* Pascal
boolean         */
#define TID_PENUM       0x29        /* Pascal
enum            */
#define TID_PWORD       0x2A        /* pword (6
byte 386 ptr) */
#define TID_TBYTE       0x2B        /* tbyte
*/
#define TID_FUNCPROTOTYPE 0x2C /* Function with
full parameter
                                    information.
*/
```

The language field is as follows:

Value	Description
0x0	Near C function
0x1	Near Pascal function
0x2	Unused
0x3	Unused
0x4	Far C function
0x5	Far Pascal function
0x6	Unused
0x7	Interrupt function

Special functions (24 bytes)

Field	Size	Offset
language	1	7
return type	4	8
class type	4	12
virtual offset	2	16

symbol index	4	18
info bits	1	22

class type is type index of class. virtual offset is offset into the virtual table. symbol index is the symbol index of this method. info bits are described in the following table.

Value	Description
0x01	member function
0x02	duplicate function
0x04	operator function
0x08	internal linkage
0x10	Pascal function passing 'this' as last parameter

```
/* Special function for methods and duplicate
functions. */
#define TID_SPECIALFUNC   0x2D
```

Classes (12 bytes)

Field	Size	Offset
class index	4	8

```
#define TID_CLASS        0x2E       /* Class
*/
```

Member pointers (24 bytes)

Field	Size	Offset
type index	4	8
class index	2	11

```
/* TID's 2F , 31-32 unused */
#define TID_HANDLEPTR   0x30    /* Handle-based
pointer NOT USED*/
#define TID_MEMBERPTR   0x33    /* Member
pointer       */
#define TID_NEWMEMPTR   0x38     /* New style
member pointer */
```

TID_MEMBERPTR

Field	Size	Offset

type index	4	8
base class index	2	12

TID_NEWMEMBERPTR

Field	Size	Offset
member ptr flags	1	7
pointer to type index	4	8
base class index	2	11

TID_HANDLEPTR

Field	Size	Offset
extra info byte	1	7
handle string index	4	8
type index	4	12

Near and far references (24 bytes)

Field	Size	Offset
type index	4	8
class index	4	12

```
#define TID_NREF        0x34        /* Near reference pointer*/
#define TID_FREF        0x35        /* Far reference pointer*/
```

```
#define TID_WORDBOOL     0x36       /* Pascal
word boolean   */
#define TID_LONGBOOL     0x37       /* Pascal
long boolean   */
#define TID_GLOBALHANDLE 0x3E       /* Windows
global handle */
#define TID_LOCALHANDLE  0x3F       /* Windows
local handle  */

/* These can be used to cast a type_rec pointer
to the appropriate
    subtype */

#define _t_pstr(x)      (((struct type_rec
*)(x))->v.pstr)
#define _t_range(x)     (((struct type_rec
*)(x))->v.range)
#define _t_bcd(x)       (((struct type_rec
*)(x))->v.bcd)
```



```
#define _t_ptr(x)       (((struct type_rec
*)(x))->v.ptr)
#define _t_seg(x)       (((struct type_rec
*)(x))->v.seg)
#define _t_carray(x)    (((struct type_rec
*)(x))->v.carray)
#define _t_vlarray(x)   (((struct type_rec
*)(x))->v.vlarray)
#define _t_parray(x)    (((struct type_rec
*)(x))->v.parray)
#define _t_struct(x)    (((struct type_rec
*)(x))->v.struc)
#define _t_vlstruct(x)  (((struct type_rec
*)(x))->v.vlstruct)
#define _t_enumty(x)    (((struct type_rec
*)(x))->v.enumty)
#define _t_function(x)  (((struct type_rec
*)(x))->v.function)
#define _t_set(x)       (((struct type_rec
*)(x))->v.set)
#define _t_bfile(x)     (((struct type_rec
*)(x))->v.bfile)
#define _t_label(x)     (((struct type_rec
*)(x))->v.label)
#define _t_specfunc(x)  (((struct type_rec
```

```
*)(x))->v.specfunc)
#define _t_class(x)     (((struct type_rec
*)(x))->v.class)
#define _t_memberptr(x) (((struct type_rec
*)(x))->v.memberptr)

struct  type_rec
{
    unsigned char   type_id;    /* The TID
byte.             */
    unsigned long   type_name;  /* Any
associated type name. */
    unsigned short  type_size;  /* The size of
any object    */
                                /*   of this
type.           */
    union
    {
        /* For TID_VOID, TID_LSTR, TID_DSTR,
TID_SQUAD,
        TID_UQUAD, TID_FLOAT, TID_PREAL,
TID_DOUBLE,
        TID_LDOUBLE, TID_BCD4,  TID_BCD8,
TID_BCD10,
        TID_ADESC, TID_LABEL, TID_TFILE,
TID_BOOL,
```



```
        TID_PWORD, TID_TBYTE types, no
additional info. */

        struct
        {   /* only for TID_PSTR */
            unsigned char max_size; /* Max
string size */
        }   pstr;
                            /*^L*/
        struct
        {
            /* for TID_PCHAR, TID_SCHAR,
TID_SINT, TID_SLONG,
            TID_UCHAR, TID_UINT and TID_ULONG
types */

            unsigned char   filler;
            unsigned long   parent; /* Parent
```

```
type   */
            long        lower;      /* Minimum
value */
            long        upper;      /* Maximum
value */
        } range;

        struct
        {   /* for TID_BCDCOB only */
            unsigned char   decimal;  /* Number
of digits to      */
                                      /*  right
of decimal point. */
        }   bcd;

        struct
        {   /* TID_LABEL only */
            unsigned char   nearfar;    /* 0
for near, 1 for far */
        }   label;

        struct
        {   /* for TID_NEAR, TID_FAR,
TID_NEAR386, TID_FAR386 */

            unsigned char   extra_info; /* as
follows:        */
            unsigned long   type_index; /*
pointed-to type    */
        }   ptr;

        /* For TID_NEAR and TID_NEAR386:

        0x0 segment register unspecified.
```



```
        0x1 ES relative
        0x2 CS relative
        0x3 SS relative
        0x4 DS relative
        0x5 FS relative
        0x6 GS relative

        For TID_FAR and TID_FAR386:

        0x0 far arithmetic.
        0x1 huge arithmetic (real mode only).
```

```
*/

        struct
        {   /* For TID_SEG, TID_NREF, TID_FREF
*/

            unsigned char   filler;
            unsigned long   type_index; /*
pointed-to type */
        }   seg;

        struct
        {   /* For TID_CARRAY only */

            unsigned char   filler;
            unsigned long   element;    /*
Element type */
        }   carray;

        struct
        {   /* For TID_VLARRAY only */

            unsigned short  upper_size; /*
Upper 16 bits of size */
            unsigned long   element;    /*
Element type          */
        }   vlarray;

        struct
        {   /* For TID_PARRAY only */

            unsigned char   filler;
            unsigned long   element;    /*
Element type   */
            unsigned short  dimension;  /*
Subscript type */
        }   parray;

        struct
        {   /* For TID_STRUCT and TID_UNION */
```



```
            unsigned char   filler;
            unsigned long   members;    /*
Index of members */
        }   struc;
```

```
        struct
        {   /* For TID_VLSTRUCT and TID_VLUNION
*/

            unsigned short  upper_size; /*
Upper 16 bits of size */
            unsigned long   members;    /*
Index of members  */
        }   vlstruct;

        struct
        {   /* For TID_ENUM and TID_PENUM */

            unsigned char   filler;
            unsigned short  parent;     /* type
of parent   */
            unsigned char   filler1;
            unsigned char   filler2;
            unsigned short  lower;      /*
Bottom of range  */
            unsigned short  upper;      /* Top
of enum range*/
            unsigned long   members;    /*
Index of members */
        }   enumty;

        struct
        {   /* For TID_FUNCTION only */

            unsigned    language : 7;
            unsigned    is_varargs : 1; /*
Accepts Var args */
            unsigned long  return_type;
        }   function;

        /*
        The language field is as follows:

        0x0 Near C function
        0x1 Near Pascal function
        0x2 Unused.
        0x3 Unused.
        0x4 Far C function
        0x5 Far Pascal function
        0x6 Unused.
        0x7 Interrupt function
        */
```

```
        struct
        {   /* For TID_FUNCPROTOTYPE only */

            unsigned    language : 7;     /*
  see TID_FUNCTION */
            unsigned    is_varargs : 1;   /*
  Accepts Var args */
            unsigned long   return_type;
            unsigned short  param_start;  /*
  starting index    */
                                          /*
  in members table */
        }   funcprototype;

        struct
        {   /* For TID_SET only */

            unsigned char   filler;
            unsigned long   parent;     /*
  Parent type */
        }   set;

        struct
        {   /* For TID_BFILE only */

            unsigned char   filler;
            unsigned short  element;    /* File
  element type*/
        }   bfile;

        struct
        {   /* For TID_SPECIALFUNC only */

            unsigned char   language;
            unsigned long   return_type;
            unsigned long   class_type;
            unsigned short  virtual_offset; /*
  in bytes */
            unsigned long   symbol_index;
            unsigned int    filler    :12;
            unsigned int    info_bits :4;
        } specfunc;

        struct
        {   /* For TID_CLASS only */

            unsigned char filler;
            unsigned short class_index;
        } class;

        struct
        {   /* For TID_MEMBERPTR */
```

```
                unsigned char  filler;
                unsigned long  type_index;
                unsigned short class_index;
            } memberptr;
        }   v;
    };
```

Members

The members table holds two completely distinct kinds of information. Structures and unions point into this table for their lists of members. Enums store their list of name/value pairs here.

Structure and union members

```
struct struct_offset_rec
{
   unsigned    filler      : 6;
       unsigned    offset_rec  : 1;
       unsigned    filler2     : 1;
       unsigned long   new_offset;
     };

     /* The new_offset is the offset for the next
     member. */

     struct  member_type
     {
       unsigned    bit_field_size  : 6;
       unsigned    offset_rec      : 1;
       unsigned    end_of_structure: 1;
       unsigned long  member_name;
       unsigned long  member_type;
     };

     /*************************************
     The member_name is the index of the name.
     The member_type is the index of the type.
     *************************************/

     struct  enum_list_type
     {
       unsigned    filler      : 7;
       unsigned    end_of_list : 1;
       unsigned long   enum_name;
       signed   short  enum_value;
     };
```

end_of_list is 1 for the last enum value in the list.

enum_name is the index of the name.

enum_value is the value of the corresponding name.

```
  typedef union
  {
    struct struct_offset_rec o;
    struct member_type       m;
    struct enum_list_type    e;
  }   member_rec;
```

bit_field_size is only important for bit field members. It is the size in bits of the member. For non-bit field members, the bit_field_size is 0.

offset_rec is zero for normal members, and non-zero for the special struct-offset record. If this bit is set, the next 2 bytes of the member record is a word holding the new structure offset in bytes. This is used for Pascal variant records.

end_of_structure is 1 for the last field in a structure. This is the sign bit, so a simple negative/non-negative test will determine the end of the structure.

Holes in the structure (due to alignment padding) are represented using an unnamed bit-field member with a zero name index and a zero type index.

The offsets of union members are always zero. The offsets of structure members are computed from the sequence of the members in the table. The members are stored in ascending offset order. For a nested unnamed union inside a structure or an unnamed structure inside a union, these will appear as unnamed members. The debugger unravels this nesting to provide functionality to support unnamed structure/union members.

Class table

```
  typedef struct {
    unsigned short parent_index; /* index into
  parent table */
    unsigned short parent_count;
    unsigned long  member_index;
    unsigned long  name_index;   /* tag */
    unsigned short virtual_ptr;  /* Offset from
  top of class data of
```



```
                                    virtual ptr*/
    unsigned char info;
                    /* Info bits:
                       bit 0:   Class is a
  virtual base class
                       bit 1:   Class is public
                       bit 2-7: Offset of method
  in virtual table */
  } class;
```

The class table defines the inheritance characteristics for each class. If a derived class has multiple inheritance, there will be multiple entries in the class table, indicating different parent classes. If there are several classes derived from the same virtual base class, there will be separate class table entries for each virtual base class, and each base class entry will have the same symbol index.

The first byte of the member record for a given class entry indicates the size of bitfields, and as a set of bits to indicate member attributes. These bits can be OR'd together to form the desired attribute.

Value	Member attributes
0x80	Last member
0x60	Static member (member_type points to symbol for the member)
0x50	Static member function

```
  0x48   Method or member function (including
         virtual and static methods)
  0x44   Virtual method
  0x42   Constructor
  0x41   Destructor
```

For example, a virtual destructor will have a value of 0x4D:

```
    0x48    - method bit
  & 0x44    - virtual bit
  & 0x41    - destructor bit
    ----
    0x4D
```


Special cases

If member_record == 0x40, record is a reset offset record.

If member_record == 0xc0, next record is a bitfield (only needed when bitfield has some of the previous attributes. Attributes are indicated in this preceding record so the first byte is free to indicate field length in the bitfield record.)

If member_record == 0x43, record is a conversion method.

If member_record == 0x80 and member_name == 0 and member_type == 0, then the Turbo Pascal linker has smart linked this class away.

Non-static, non-bitfield data members are always 0, or 0x80 if they're the last item.

Bit combining doesn't apply to constructors, destructors and conversions bits, since they are mutually exclusive.

Parent table

Each entry in the parent table has the following format:

```
  typedef struct
  {
    unsigned short class_index;       /* index
  into class table */
  } parent;
```

class_index is an index into the class table. If the highest bit is set, this parent is a virtual base class.

Scope class table

```
  typedef struct
  {
      unsigned short class_index;       /* index
  into class table */
      unsigned short class_count;       /* number
  of classes      */
  } scope_class;
```


A scope class table finds the classes defined within a particular scope. If any scope class records are needed, there must be one record for each scope record. This is identical to expanding the current scope record to contain the following fields, but it maintains backward compatibility with the earlier table, and allows non-object languages to avoid the overhead of bigger scope records.

Module class table

```
  typedef struct                    /* local
  classes          */
  {
      unsigned short class_index;   /* index into
```

```
  class table */
      unsigned short class_count;   /* number of
  classes      */
  } module_class;
```

A module class table finds the classes and overloads defined within a particular module. If any module class records are needed, there must be one for each module record. This is identical to expanding the current module record to contain the following fields, but it maintains backward compatibility with the earlier table, and allows non-object languages to avoid the overhead of bigger module records.

Coverage offset map table

```
  typedef struct
  {
      unsigned short offset;  /* index into
  Coverage Offset Table */
  } TCoverageOffsetMapTableEntry;
```

This table defines the starting index into the coverage offset table (which follows) for the given segment. There are as many segment entries as there are segments in the segment table. This table can be viewed as an array of TCoverageOffsetMapRecord entries, with the number of entries the same as the number of segments records in the segment table. Entries with an index of 0 indicate that lack of coverage offsets for the given segment. Note that the values in

this table are not necessarily in ascending order.

Coverage offset table

```
  typedef struct
  {
      unsigned short offset;        /* offset
  into segment */
```

```
  } TCoverageOffsetTableEntry;
```

Each entry in the table corresponds to a starting offset for a block of code that is "atomic," meaning that if you start executing at the beginning of the block, you are guaranteed to reach the end.

Browser definition table

```
  struct TDefinitionRecord
  {
    unsigned long  symbol_index;  /* The index of
  the symbol in   */
                                  /* the Symbols
  table            */
    unsigned short file_index;    /* Which file
  the symbol is in  */
    unsigned short line_number;   /* line number
  in the file      */
  };
```

Optimized symbol table

```
  struct opt_symbol_record {
      unsigned short  opt_symbol_next;
                      /* index to next record for
  this symbol */
      unsigned short  opt_symbol_offset;
                      /* offset is treated as a
  register enum */
                      /* See the Symbols section
  for details  */
      unsigned char   opt_symbol_class;
                      /* Interpreted as for
  symbol_record     */
      unsigned short
  opt_symbol_code_offset_start;
                      /* start of optimization
  range          */
```



```
      unsigned short  opt_symbol_code_offset_end;
  };                  /* end of optimization
  range            */
```

An has an entry in the symbols table whose type is SC_REGISTER (0x4), but whose register ID (offset) is greater than or equal to 0x28. The register ID (minus 0x28) is an index into the optimized symbols table. The at that index is the first record in a linked list of records, linked through the opt_symbol_next field. The end of the list is marked by a 0 in that field. This record will have accurate information as to the true location of the variable in the opt_symbol_offset and opt_symbol_class fields, as per the symbol_record specification. Note that opt_symbol_class refers to the combination of the three symbol record bit fields: symbol_class, has_valid_BP, and return_address_word_offset.

The reason there is a list of opt_symbol_record objects is that a variable may exist in a register for some period of time, and then be "spilled" to a memory location, and possibly later reloaded into another register.

Module Optimization Flags Table, Reference Information Table

The DebugFlags field in the debug header extension currently have only one bit defined:

```
#define DBG_OPT         0x0001
```

If this bit is set, then the application has optimized code somewhere in its modules. The ModuleFlags table contains a dword entry of flags for each module in the Module table. It is indexed by the same module index that is used to index the module table.

Note that the optimizations performed may be different than the optimizations requested when the module was compiled.

Each word currently describes the sorts of optimizations the compiler has done to the module. The following bits are defined:

```
#define MO_globalCSEs           0x0001
#define MO_localCSEs            0x0002
#define MO_inductVars           0x0004
```

```
#define MO_codeMotion           0x0008
#define MO_regAlloc             0x0010
#define MO_loadOptim            0x0020
#define MO_loopOpt              0x0040
#define MO_intrinsics           0x0080
#define MO_deadStorElim         0x0100
#define MO_copyProp             0x0200
#define MO_jumpOpt              0x0400
#define MO_speed_size           0x0800
#define MO_noAliasing           0x1000
```

If the dword is 0, then the module contains no optimized code.

Reference Information Table

Names

Any symbolic name encountered in the symbol tables is referenced via an index into this region. Each identifier is stored with a trailing null byte.

Debugging Turbo Pascal overlays

Data at address pointed to by debugger_hook:

```
typedef struct
{
  unsigned short overlay_list; /* start of
linked list of overlay */
                               /* header segs
*/
  unsigned short overlay_size; /* smallest
overlay buffer that    */
                               /* can be used
*/
  void far *  debugger_hook;   /* ptr to
routine in debugger      */
} overlay;
```

A debugger must fill in debugger_hook after loading the program. debugger_hook is called by the overlay manager after any overlay is loaded. The allows the debugger to set in the newly loaded segment. When called, ES contains the base segment of the overlay header BX contains the offset that the overlay manager will jump to in the newly loaded code. (This is useful if an int 3F has been traced--an int 3f is followed by data

and is not returned.)

The actual segment of a particular overlaid segment is at offset 10h in the overlay header. If this value is zero, then the segment is not loaded.

Data objects in an overlaid segment will contain the segment of the overlay header and the true offset in the code segment.

CHAPTER

4

Project file format

You can view a project file directly with a debugger or binary editor but the Project File utilities make it a lot easier to understand and work with. This chapter describes the utilities and gives information for the Turbo C++ and Borland C++ project file format. The format is current as of Project file version 0x0701.

Project file utilities

How the utilities work

Using object oriented technology, the online utilities provide access to project (.PRJ) files produced by Turbo C++ and Borland C++. The examples PROX, STRIPPRJ, and TRANCOPY show how you can see and change project files without needing to learn how the data is organized.

Two basic classes access the project files. TFileClass gets to files on disk (see fileclas.h and FILECLAS.CPP). TSection and descendants encapsulate each section of a project (see prjclass.h and PRJCLASS.CPP).

A project can be divided into seven discrete sections, each storing different information. PROX defines them as classes. For example, TOptionSection contains the settings of many options, such as Options|Compiler|Code generation|Model. Here's the TSection class hierarchy and contents:

TSection

├─TOptionSection	Compiler, linker, and other information shown in the Options menu
├─THeaderSection	Date and time of the project
├─TTransferSection	Information shown in Options|Transfer
├─TNoteSection	Contents of Window|Project note
├─TModuleSection	Contents of Project Window
├─TDependencySection	Contents of Project|View includes
└─TExtensionSection	Miscellaneous string contents of Project|Local Options, referenced by TModuleSection

TSection's derived classes have member functions to access their unique data in the most convenient way. For example,

TModuleSection::GetModule returns a pointer to a structure containing information on the specified module. TOptionSection::GetCompilerModel returns the setting of the memory model.

The following table shows which examples explore a given section:

Project Section	PROX	TRANCOPY	STRIPPRJ
OptionSection	X		
HeaderSection	X		
TransferSection		X	
NoteSection	X		
ModuleSection	X		X
DependencySection	X		X
ExtensionSection	X		

Using the examples

To learn how to use the core classes, study the code in the project file utilities, and try the examples. With the source code, you can use the debugger to trace them. Start with PROX, a collection of small, separate functions that perform a variety of tasks. Use PROX.PRJ as your source. PROX's syntax is:

```
PROX [options] <filename> [.PRJ] [options]
```


Show overview (-o)

Shows the file offset and size of each section. The Dependency section is missing until files are included during compilation.

Show modules (-p)

Shows each item seen in the Project Window, along with Local Options such as the output name, command line overrides, translator, and whether or not debug info is excluded. When used on a complete project file, it teaches how to access

the Module section of a project file using TModuleSection. It also demonstrates that each module may have an index to the Extensions section, stored in TExtensionSection, which contains additional strings for the output path, command line overrides, and translator when used with a project that contains Local Options.

Show modules with dependencies (-P)

Same as -p except shows the include files (dependencies) of each module, stored in TDependencySection.

Show options (-t)

Displays memory model, prolog/epilog, paths, and other selected options stored in TOptionSection.

Set options (-s)

Modifies and writes memory model, prolog/epilog, paths, and other selected options stored in TOptionSection. Writes these changes to FOO.PRJ. You can open FOO.PRJ in the IDE to verify the modifications. However, do not use the project for actual work, as the options are not valid.

Show note (-n)

Shows Window | Project Note using TNoteSection.

Show header (-h)

Outputs the age of the project using THeaderSection. Shows the date and time in ASCII, not hexadecimal.

TRANCOPY syntax

TRANCOPY [-r] <source project> <destination project>

Using PROX helps you understand most of the project. However, PROX totally ignores TTransferSection. With TRANCOPY you can copy the transfer section of one project into another project. Without the -r option, the source section is nondestructively merged into the

destination section. With the -r option, the previous transfer items are replaced. The TRANCOPY executable ships with both Turbo C++ for Windows and Borland C++.

STRIPPRJ syntax

STRIPPRJ <source project> <destination project>

STRIPPRJ removes include file information (the Dependency section) from a project. It covers the same areas as PROX -P and PROX -s. You can regenerate the Dependency section by performing Compile|Build all.

Format of the Project file

Header
Option section
Header section
Transfer section
Note section
Module section
Dependency section
Extension section
-1 (0xFFFF)

If you use the Project file utilities you probably won't have to learn the Project file format. The class hierarchy does most of the work for you. The rest of this chapter documents the format for direct access.

The first part of the .PRJ file is Header information used by the IDE to confirm the file's

validity. The following seven sections differ in structure and kinds of information they contain. However, they each have a section header to identify Block Type and size of the data area.

Viewing .PRJ files is difficult. You must carefully track offsets to be sure you have the right data. If you are just getting started, you might follow the example. First use PROX -o PROX.PRJ and record the offset for each section. Type TD to enter the Turbo Debugger IDE and choose View|File|PROX.PRJ.

Header information
variable length: VisibleIDString = "Turbo C Project File ^Z"
String designed to display if the project file is listed to the screen (null terminated).

7 bytes: Signature = "01 0D 12 17 01 1A 00"
ID number that the IDE verifies .

2 bytes: Version
Unsigned version number that is written into the project file when it is created. For internal use. The version number changes whenever any change occurs in either the project file format or data. This version must match that held in the IDE, or the project manager will not accept the file. The current version is 0x0701. In the file, the number reads 01 07 due to byte swapping.

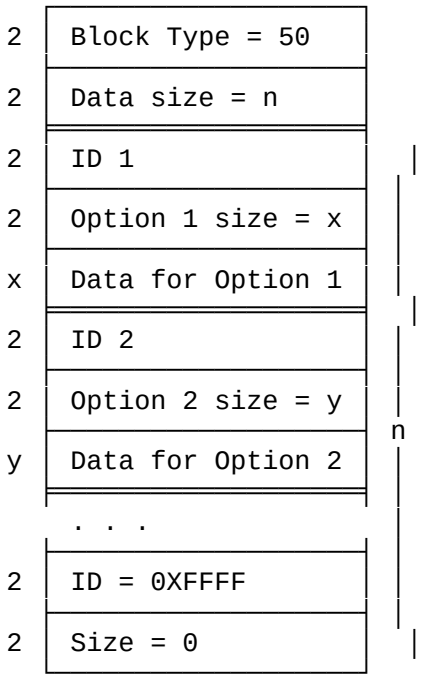
Sections in the project file
Each section begins with a section header as follows:

2 bytes: section Block Type identification number

2 bytes: size of the following data area in the section

The Block Types are given here in decimal values. The size of block does not include the 4-byte header. Here are the sections that make up a project file.

Block Type 50--
Options section



variable length data: array of structures. For each Options menu item:

2 bytes: Option ID

2 bytes: size of Option

variable length data: value, data, or content of Option

The structure for each Options menu item has a 4-byte header followed by the data, or content or the item. The last ID is 0xFFFF with a size of 0. You can write to Block Type 50 (32 00 in the file).

Block Type 51--
Header section

2	Block Type = 51
2	Size = 6
2	Reserved
4	Project age

2 bytes: Reserved

4 bytes: Age of project file =

seconds: 5 bits

minutes: 6 bits

hour: 5 bits

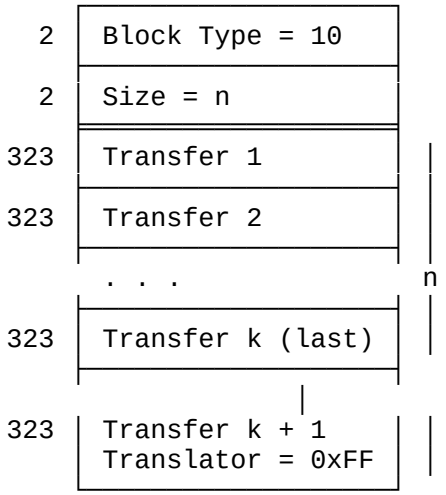
day: 5 bits

month: 4 bits

year: 7 bits

Block Type 51 (33 00 in the file) is used internally.

Block Type 10--
Transfer section



variable length data: array of structures. For each Options|Transfer item:

1 byte: translator[]; 1=true, 0=false, 0xFF is last;

40 bytes: transfer title (Name)

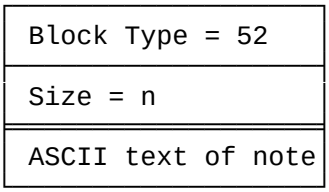
80 bytes: transfer exe name (Program path)

200 bytes: transfer command (Command line)

2 bytes: Hot key command

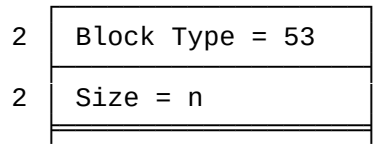
After the header, the total number of bytes used is a multiple of 323 (depending on how many transfer items are included). You can write to Block Type 10 (0a 00 in the file).

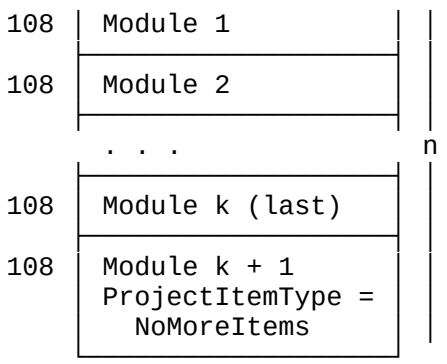
Block Type 52--Note section



variable length data after the header. You can edit the note in Block Type 52 (34 00 in the file).

Block Type 53--Module section





variable length data: each module represents an item in the Project Window, structured as follows:

2 bytes: ProjectItemType =

reserved 0x0001

reserved 0x0002

Translator 0x0004

Overlay 0x0008 (Project window Options|Local Options)

CommandLineOverride 0x0010 (Local Options)

Exclude Debug info 0x0020 (Local Options)

Exclude from link 0x0040 (Local Options)

No more items 0x8000 (= 1, TRUE if is last item)

2 bytes: DependencyID index into Block Type 54

See Block Type 51 age bits.

4 bytes: Obj age (0 if not available)

4 bytes: Code Size (-1 if not available)

4 bytes: Data Size (-1 if not available)

2 bytes: number of lines

2 bytes: reserved: (= 0)

80 bytes: filename of item

See Block Type 55 for use.

2 bytes: Options enum index into Block Type 55 (Local Options|Command-Line Options)

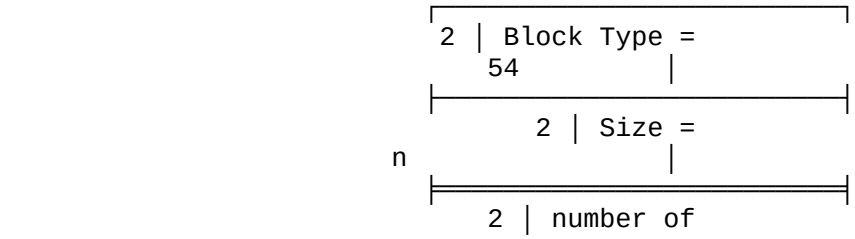
2 bytes: Translator Title index into Block Type 55 (Local Options|Translator)

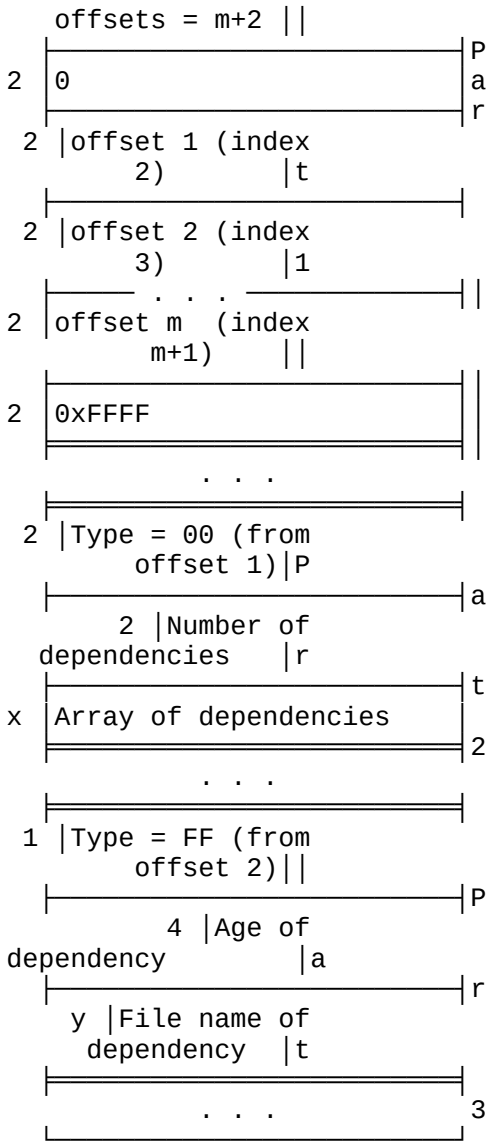
2 bytes: OutputName index into Block Type 55 (Local Options output path)

2 bytes: Reserved

You can write to unreserved parts of Block Type 53 (35 00 in the file).

Block Type 54--
Dependency section





A memory manager creates the Dependency section containing pointers to include files, which is complex yet efficient. The data area starts after the 4-byte header. It consists of three variable length parts (basically offsets, indexes, and include files) as follows:

Part 1. Offsets

variable length data: array of 2-byte integers containing offsets from the beginning of the data area directly following the 4-byte header. The number of offsets is the first element. See the diagram for the rest of the array content.

Part 2. Module dependencies

variable length data: type, number of entries, and array of dependencies for each module in the project:

2 bytes: Type = 00 00

2 bytes: Number of dependency entries (multiple of 4)

variable length data, for each dependency:

2 bytes: index to array of offsets in part 1. The last entry is -1.

4 bytes: age when dependency last compiled for this module. See Block Type 51 for age bits.

Part 3. Dependency information

variable length data: series of bytes containing type, age, and file name for each dependency:

1 byte: Type = FF

4 bytes: age of dependency (see Block Type 51 for age bits)

variable length string: file name of dependency, NULL terminated

You can write to unreserved parts of Block Type 54 (36 00 in the file).

Here are some tips for tracking a dependency entry in a Project file, FILENAME.PRJ.

Prepare as follows:

1. Run PROX -o FILENAME.PRJ to make note of the project file offsets of the Module and Dependency sections.

2. Enter TD and open the file under View|File|Open.

Get the Dependency ID offset as follows:

1. Locate the Module section offset (35 00 value).

2. Count four bytes, skipping over the header.

3. Count two bytes, skipping over the Project item type.

4. Record the 2-byte Dependency ID offset.

Find the Module dependency entry:

1. Locate the Dependency section offset (36 00 value).

2. Count four bytes to the start of the data area.

3. Count 2* Dependency ID offset to read the offset to the Module dependencies. See Part 1 on the diagram.

4. Return to the start of the data area.

5. Count off the Module dependencies offset.

6. You should be at a Type 00 00 location. See Part 2 on the diagram.

Find the dependency information:

1. Skip over 4 bytes for the header.

2. Read the index.

3. Go to the beginning of the data area.

4. Count 2*index.

5. Read offset of Dependency information.

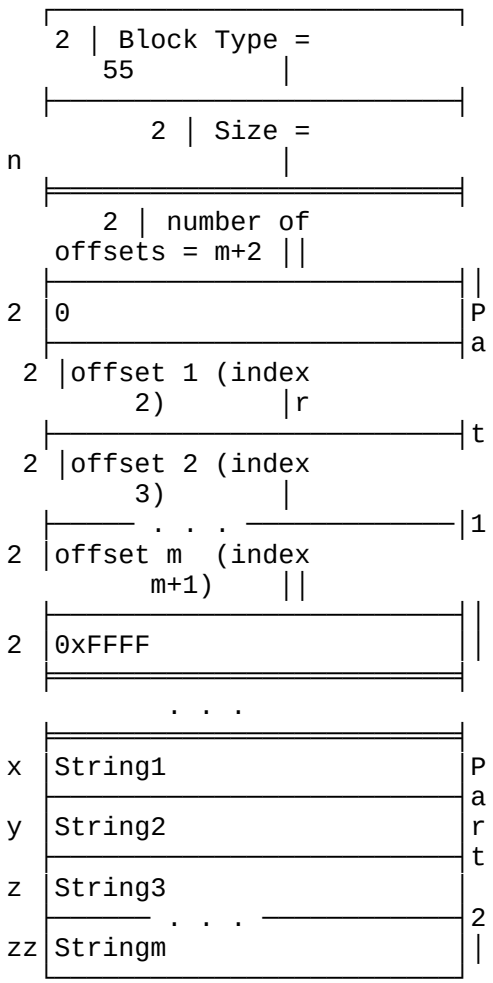
6. Go to this offset. See Part 3 on the diagram.

7. Skip 5 bytes past the type and age data.

8. Read the file name (NULL terminated).

For each dependency, read the index (separated from the previous one by 4 bytes of age data) and repeat steps 3-6. The part ends with 0xFFFF.

Block Type 55--
Extension section



This is entered with an index into an integer

array, obtained from the Options, Translator

Title, and OutputName fields of each module in Block Type 53.

Here are some tips for tracking Options, Translator Title, and OutputName entries for a module in a Project file, FILENAME.PRJ.

Prepare as follows:

1. Run PROX -o FILENAME.PRJ to make note of the project file offsets of the Module and Extension sections.

2. Enter TD and open the file under View|File|Open.

Get the Options, Translator, and OutputName offsets as follows:

1. Locate the Module section offset (35 00 value).

2. Count 4 bytes, skipping the header.

3. Count 100 bytes.

4. Record the next three 2-byte Options, Translator, and OutputName offsets.

Find the entries:

1. Locate the Extension section offset (37 00 value).

2. Count four bytes to the start of the data area.

3. Count 2* Options offset to read the offset to the string. See Part 1 on the diagram.

4. Return to the start of the data area.

5. Count off the string's offset.

6. Read the string. See Part 2 on the diagram.

CHAPTER

5

The BGI driver toolkit

The Borland Graphics Interface (BGI) is a fast, compact, and device-independent software package for graphics development built into the Turbo Pascal and Borland C++, language products. Device independence is achieved via loadable device-specific drivers called from a common kernel. In this chapter we describe basic BGI functionality, and how to create new device drivers.

File Name	File Description
BH.C	BGI loader header-building program source
BH.EXE	BGI loader header-building program executable
DEVICE.INC	Structure and macro definition file
DEBVECT.ASM	Vector table for sample (DEBUG) driver
DEBUG.C	Main module for sample driver
MAKEFILE	Build file
BUILD.BAT	A batch file for MAKE-phobics

BGI run-time architecture

Programs produced by Borland languages create graphics via two entities acting in concert: the generic BGI Kernel and a device-specific driver. Typically, an application built with a Borland compiler will include several device driver files on the distribution disk (extension .BGI) so that the program can run on various types of screens and printers. Graphics requests (for example, draw line, draw bar, etc.) are sent by the application to the BGI Kernel, which in turn makes requests of the device driver to actually manipulate the hardware.

A BGI device driver is a binary image; that is, a sequence of bytes without symbols or other linking information. The driver begins with a short header, followed by a vector table containing the entry points to the functions inside. The balance of the driver comprises the code and data required to manipulate the target graphics hardware.

All code and data references in the driver must be near (i.e., small model, offset only), and the entire driver, both code and data, must fit within 64K. In

use, the device driver can count on its being loaded on a paragraph boundary.

The BGI Kernel uses a register-based calling convention to communicate with the device driver (described in detail below).

BGI Graphics Model

When considering the functions listed here, keep in mind that BGI performs most drawing operations using an implicit drawing or tracing color (COLOR), fill color (FILLCOLOR), and pattern (FILLPATTERN). For example, the PIESLICE call accepts no pattern or color information, but instead uses the previously set COLOR value to trace the edge of the slice, and the previously set FILLCOLOR and FILLPATTERN values for the interior.

For efficiency, many operations take place at the position of the current pointer, or CP. For example, the LINE routine accepts only a single (x,y) coordinate pair, using the CP as the starting point of the line and the passed coordinate pair as the ending point. Many functions (LINE, to name one) affect CP, and the MOVE function can be used to explicitly adjust CP. The BGI coordinate system places the origin (pixel 0,0) at the upper left-hand corner of the screen.

Header Section

The device header section, which must be at the beginning of the device driver, is built using macro BGI defined in file DEVICE.INC. The BGI macro takes the name of the device driver to be built as an argument. For example, a driver named DEBUG would begin as shown here:

```
  CSEG   SEGMENT PARA PUBLIC 'CODE'  ; any segment naming may be used
         ASSUME  DS:CSEG, CS:CSEG    ; cs=ds

         CODESEG

         INCLUDE DEVICE.INC          ; include the device.inc file
         BGI     DEBUG               ; declare the device header section
```

The device header section declares a special entry point known as EMULATE. If the action of a device driver vector is not supported by the

hardware of a device, the vector entry should contain the entry EMULATE. This will be patched at load time to contain a jump to the Kernel's emulation routine. These routines will emulate the action of the vector by breaking down the request into simpler primitives. For example, if the hardware has the functionality to draw arc, the arc vector will contain the address of the routine to dispatch the arc data to the hardware and would appear as follows:

```
  dw      offset ARC    ; Vector to the arc routine
```

If, as is often the case, the hardware doesn't have the functionality to display arcs, the vector would instead contain the EMULATE vector:


```
  dw     EMULATE
```

The Kernel has emulation support for the following vectors:

BAR	Filling 3D rectangles
ARC	Elliptical arc rendering
PIESLICE	Elliptical pie slices
FILLED_ELLIPSE	Filled Ellipses

The driver status table

BGI requires that each driver contain a Driver Status Table (DST) to determine the basic characteristics of the device that the driver addresses. As an example, the DST for a CGA display is shown here:

```
  STATUS STRUC
  STAT    DB     0          ; Current Device Status (0 = No Errors)
  DEVTYP  DB     0          ; Device Type Identifier (must be 0)
  XRES    DW     639        ; Device Full Resolution in X Direction
  YRES    DW     199        ; Device Full Resolution in Y Direction
  XEFRES  DW     639        ; Device Effective X Resolution
  YEFRES  DW     199        ; Device Effective Y Resolution
  XINCH   DW     9000       ; Device X Size in inches*1000
```

```
  YINCH   DW     7000        ; Device Y Size in inches*1000
  ASPEC   DW     4500        ; Aspect Ratio = (y_size/x_size) *
10000
          DB     8h
          DB     8h          ; for compatibility, use these values
          DB     90h
          DB     90h
  STATUS  ENDS
```

The BGI interface provides a system for reporting errors to the BGI Kernel and to the higher level code developed using Borland's language packages. This is done using the STAT field of the Driver Status Table. This field should be filled in by the driver code if an error is detected during the execution of the device installation (INSTALL). The following error codes are predefined in include file GRAPHICS.H for Turbo C and in the Graphics unit for Turbo Pascal.

```
  grOk              =   0          Normal Operation, No errors
  grNoInitGraph     =  -1
  grNotDetected     =  -2
  grFileNotFound    =  -3
  grInvalidDriver   =  -4
  grNoLoadMem       =  -5
  grNoScanMem       =  -6
  grNoFloodMem      =  -7
  grFontNotFound    =  -8
  grNoFontMem       =  -9
  grInvalidMode     = -10
  grError           = -11          Generic Driver Error
  grIOerror         = -12
```



```
  grInvalidFont     = -13
  grInvalidFontNum  = -14
  grInvalidDeviceNum = -15
```

The next field in the Device Status Table, DEVTYP, describes the class of the device that the driver controls; for screen devices, this value is always 0.

The next four fields, XRES, YRES, XEFRES, and YEFRES, contain the

number of
pixels available to BGI on this device in the horizontal and vertical
dimensions, minus one. For screen devices, XRES=XEFRES and YRES=YEFRES. The
XINCH and YINCH fields are the number of inches horizontally and vertically into
which the device's pixels are mapped, times 1000. These fields in conjunction
with XRES and YRES permit device resolution (DPI, or dots per inch) calculation.

Horizontal resolution (DPI) = (XRES+1) / (XINCH/1000)
Vertical resolution (DPI) = (YRES+1) / (YINCH/1000)

The ASPEC (aspect ratio) field is effectively a multiplier/divisor pair (the
divisor is always 10000) that is applied to Y coordinate values to produce
aspect-ratio adjusted images (for example, round circles). For example, an ASPEC
field of 4500 implies that the application will have to transform Y coordinates
by the ratio 4500/10000 when drawing circles to that device if it expects them
to be round. Individual monitor variations may require an additional adjustment
by the application.

The device driver vector table

The routines in the device driver are accessed via a vector table. This table is
at the beginning of the driver and contains 16-bit offsets to subroutines and
configuration tables within the driver. The format of the vector table is shown
below.

```
  VECTOR_TABLE:

          DW      INSTALL   ; Driver initialization and installation
          DW      INIT      ; Initialize device for output
          DW      CLEAR     ; Clear graphics device; get fresh screen
          DW      POST      ; Exit from graphics mode, unload plotter
          DW      MOVE      ; Move Current Pointer (CP) to (X,Y)
          DW      DRAW      ; Draw Line from (CP) to (X,Y)
          DW      VECT      ; Draw line from (X0,Y0) to (X1,Y1)
          DW      EMULATE   ; Reserved, must contain Emulate vector
          DW      BAR       ; Filled 3D bar from (CP) to (X,Y)
          DW      PATBAR    ; Patterned rectangle from (X,Y) to (X1,Y1)
          DW      ARC       ; Define ARC
          DW      PIESLICE  ; Define an elliptical pie slice
          DW      FILLED_ELLIPSE  ; Draw a filled ellipse
          DW      PALETTE   ; Load a palette entry
```

```
                DW      ALLPALETTE  ; Load the full palette
                DW      COLOR     ; Set current drawing color/background
                DW      FILLSTYLE ; Filling control and style
```



```
                DW      LINESTYLE ; Line drawing style control
                DW      TEXTSTYLE ; Hardware Font control
                DW      TEXT      ; Hardware Draw text at (CP)
                DW      TEXTSIZ   ; Hardware Font size query
                DW      RESERVED  ; Reserved
                DW      FLOODFILL ; Fill a bounded region
                DW      GETPIX    ; Read a pixel from (X,Y)
                DW      PUTPIX    ; Write a pixel to (X,Y)
                DW      BITMAPUTIL ; Bitmap Size query function
                DW      SAVEBITMAP ; BITBLT from screen to system memory
                DW      RESTOREBITMAP ; BITBLT from system memory to
screen
                DW      SETCLIP    ; Define a clipping rectangle
                DW      COLOR_QUERY  ; Color Table Information Query
        ;
        ;       35 additional vectors are reserved for Borland's future
use.
        ;
                DW      RESERVED     ; Reserved for Borland's use (1)
                DW      RESERVED     ; Reserved for Borland's use (2)
                DW      RESERVED     ; Reserved for Borland's use (3)
                .
                .
                .
                DW      RESERVED     ; Reserved for Borland's use (33)
                DW      RESERVED     ; Reserved for Borland's use (34)
                DW      RESERVED     ; Reserved for Borland's use (35)
        ;
        ;       Any vectors following this block may be used by
        ;       independent device driver developers as they see fit.
        ;
```

Vector Descriptions

The following information describes the input, output, and function of each of the functions accessed through the device vector table.

```
  dw   offset INSTALL      ; device driver installation
```

The Kernel calls the INSTALL vector to prepare the device driver for

use. A
function code is passed in AL. The following function codes are
defined:

```
  >>> Install Device:   AL = 00
    Input:
       CL = Mode Number for device

    Return:
       ES:BX --> Device Status Table (see STATUS structure)
```

The INSTALL function is intended to inform the driver of the
operating
parameters that will be used. The device should not be switched to
graphics mode
(see INIT). On input, CL contains the mode in which the device will
operate.
(refer to BGI setgraphmode statement)

The return value from the Install Device function is a pointer to a
Device
Status Table (described earlier).

```
  >>> Mode Query:   AL = 001h
    Input:
       Nothing

    Return:
       CX  The number of modes supported by this device.
```

The MODE QUERY function inquires about the maximum number of modes
supported by
this device driver.

```
  >>> Mode Names:   AL = 002h
    Input:
       CX  The mode number for the query.

    Return:
       ES:BX --> a Pascal string containing the name
```

The MODE NAMES function inquires about the ASCII form of the mode
number present
in CX. The return value in ES:BX points to a Pascal string
describing the given
mode. (Note: A Pascal, or _length_, string is a string in which the

first byte
of data is the number of characters in the string, followed by the string data
itself.) To ease access to these strings from C, the strings should be followed
by a zero byte, although this zero byte should not be included in the string
length. The following is an example of this format:

```
NAME:      db       16, '1280 x 1024 Mode', 0
```

==========

```
DW   offset   INIT            ; Initialize device for output
```

Input:
ES:BX --> Device Information Table

Return:
Nothing

This vector changes an already INSTALLed device from text mode to graphics mode.
This vector should also initialize any default palettes and drawing mode
information as required. The input to this vector is a device information table
(DIT). The format of the DIT is shown below and contains the background color
and an initialization flag. If the device requires additional information at
INIT time, these values can be appended to the DIT. There in no return value for
this function. If an error occurs during device initialization, the STAT field
of the Device Status Table should be loaded with the appropriate error value.

```
; ************** Device Information Table Definition **************
```



```
struct   DIT
         DB     0         ; Background color for initializing
```

```
screen
                        DB     0          ; Init flag; 0A5h = don't init; anything
                                          ; else = init
                        DB     64 dup 0   ; Reserved for Borland's future use
                                          ; additional user information here
                DIT     ends


=================================================================

                        DW   offset   CLEAR  ; Clear the graphics device

                    Input:
                       Nothing

                    Return:
                       Nothing

                This vector clears the graphics device to a known state. In the case
of a CRT
                device, the screen is cleared. In the case of a printer or plotter,
the paper is
                advanced, and pens are returned to the station.

                        DW   offset   POST       ; Exit from graphics mode

                  Input:
                     Nothing

                  Return:
                     Nothing

                This routine closes the graphics system. In the case of graphics
screens or
                printers, the mode should be returned to text mode. For plotters,
the paper
                should be unloaded and the pens should be returned to station.

                        DW   offset   MOVE   ; Move the current drawing pointer

                  Input:
                     AX         the new CP x coordinate
                     BX         the new CP y coordinate

                  Return:
                     Nothing

                Sets the Driver's current pointer (CP) to (AX,BX). This function is
used prior
                to any of the TEXT, ARC, SYMBOL, DRAW, FLOODFILL, BAR, or PIESLICE
routines to
                set the position where drawing is to take place.

                        DW   offset   DRAW        ; Draw a line from the (CP) to (X,Y)

                  Input:
                     AX         The ending x coordinate for the line
```

```
        BX          The ending y coordinate for the line

    Return:
        Nothing
```

Draws a line from the CP to (X,Y). The current LINESTYLE setting is used. The current pointer (CP) is updated to the line's endpoint.

```
        DW          VECT            ; Draw line from (X1,Y1) to (X2,Y2)

    Input:
        AX          X1; The beginning X coordinate for the line
        BX          Y1; The beginning Y coordinate for the line
        CX          X2; The ending X coordinate for the line
        DX          Y2; The ending Y coordinate for the line

    Return:
        Nothing
```

Draws a line from the (X1,Y1) to (X2,Y2). The current LINESTYLE setting is used to draw the line. Note: CP is NOT changed by this vector.

```
            DW      BAR         ; fill and outline rectangle (CP),(X,Y)

    Input:
        AX          X--right edge of rectangle
        BX          Y--bottom edge of rectangle
        CX          3D = width of 3D bar (ht := .75 * wdt); 0 = no 3D effect
        DX         3D bar top flag; if CX <> 0, and DX = 0, draw a top

    Return:
        Nothing
```

Fills and outlines a bar (rectangle) using the current COLOR, FILLCOLOR, and FILLPATERN. The current pointer defines the upper left corner of the rectangle and (X,Y) is lower right. An optional 3D shadow effect (intended for business graphics programs) is obtained by making CX nonzero. DX then serves as a flag indicating whether a top should be drawn on the bar.

```
        DW     PATBAR       ; fill rectangle (X1,Y1), (X2,Y2)

  Input:
     AX        X1--the rectangle's left coordinate
     BX        Y1--the rectangle's top coordinate
     CX        X2--the rectangle's right coordinate
     DX        Y2--the rectangle's bottom coordinate

  Return:
     Nothing
```

Fills (but doesn't outline) the indicated rectangle with the current fill pattern and fill color.


```
        DW       ARC          ; Draw an elliptical arc

  Input:
     AX         The starting angle of the arc in degrees (0-360)
     BX         The ending angle of the arc in degrees (0-360)
     CX         X radius of the elliptical arc
     DX         Y radius of the elliptical arc

  Return:
     Nothing
```

ARC draws an elliptical arc using the (CP) as the center point of the arc, from the given start angle to the given end angle. To get circular arcs the application (not the driver) must adjust the Y radius as follows:

```
  YRAD := XRAD * (ASPEC / 10000)
```

where ASPEC is the aspect value stored in the DST.

```
        DW      PIESLICE        ; Draw an elliptical pie slice

  Input:
     AX         The starting angle of the slice in degrees (0-360)
     BX         The ending angle of the slice in degrees (0-360)
     CX         X radius of the elliptical slice
     DX         Y radius of the elliptical slice

  Return:
     Nothing
```

PIESLICE draws a filled elliptical pie slice (or wedge) using CP as the center of the slice, from the given start angle to the given end angle. The current FILLPATTERN and FILLCOLOR is used to fill the slice and it is outlined in the current COLOR. To get circular pie slices, the application (not the driver) must adjust the Y radius as follows:

```
YRAD := XRAD * ASPEC / 10000
```

where ASPEC is the aspect value stored in the driver's DST.

```
        DW      FILLED_ELLIPSE   ; Draw a filled ellipse at (CP)

  Input:
     AX         X Radius of the ellipse
     BX         Y Radius of the ellipse

  Return:
     Nothing
```

This vector draws a filled ellipse. The center point of the ellipse is assumed to be at the current pointer (CP). The AX Register contains the X Radius of the ellipse, and the BX Register contains the Y Radius of the ellipse.


```
        DW      PALETTE       ; Load a color entry into the Palette

  Input:
     AX         The index number and function code for load
     BX         The color value to load into the palette

  Return:
     Nothing
```

The PALETTE vector loads single entries into the palette. The register AX contains the function code for the load action and the index of the color table entry to be loaded. The upper two bits of AX determine the action to be taken. The table below tabulates the actions. If the control bits are 00,

the color
table index in (AX AND 03FFFh) is loaded with the value in BX. If
the control
bits are 10, the color table index in (AX AND 03FFFh) is loaded with
the RGB
value in (Red=BX, Green=CX, and Blue=DX). If the control bits are
11, the color
table entry for the background is loaded with the value in BX.

Control Bits	Color Value and Index
00	Register BX contains color, AX is index
01	not used
10	Red=BX Green=CX Blue=DX, AX is index
11	Register BX contains color for background

==========

```
       DW       ALLPALETTE         ; Load the full palette
```

Input:
ES:BX --> array of palette entries

Return:
Nothing

The ALLPALETTE routine loads the entire palette in one driver call.
The register
pair ES:BX points to the table of values to be loaded into the
palette. The
number of entries is determined by the color entries in the Driver
Status Table.
The background color is not explicitly loaded with this command.

```
       DW       COLOR          ; Load the current drawing color.
```

Input:

AL	The index number of the current drawing color
AH	The index number of the fill color

Return:
Nothing

The COLOR vector determines the current drawing color. The value in
AL is the
index into the palette of the new current drawing color. The value
in the AH

register is the color index of the new fill color. All primitives are drawn with the current drawing color until the color is changed.

The fill color is used for the interior color for the bar, polygons, pie slice, and floodfill primitives.

==========

```
        DW      FILLSTYLE   ; Set the filling pattern
```

Input:
- AL — Primary fill pattern number
- ES:BX — If pattern number is 0FFh, points to user-defined pattern mask.

Return:
- Nothing

Sets the fill pattern for drawing. The fill pattern is used to fill all bounded regions (BAR, POLY, and PIESLICE). The numbers for the predefined fill patterns are as follows:

Code	Description	8 Byte fill pattern
0	No Fill	000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
1	Solid Fill	0FFh, 0FFh, 0FFh, 0FFh, 0FFh, 0FFh, 0FFh, 0FFh
2	Line Fill	0FFh, 0FFh, 000h, 000h, 0FFh, 0FFh, 000h, 000h
3	Lt Slash Fill	001h, 002h, 004h, 008h, 010h, 020h, 040h, 080h
4	Slash Fill	0E0h, 0C1h, 083h, 007h, 00Eh, 01Ch, 038h, 070h
5	Backslash Fill	0F0h, 078h, 03Ch, 01Eh, 00Fh, 087h, 0C3h, 0E1h
6	Lt Bkslash Fill	0A5h, 0D2h, 069h, 0B4h, 05Ah, 02Dh, 096h, 04Bh
7	Hatch Fill	0FFh, 088h, 088h, 088h, 0FFh, 088h, 088h, 088h
8	XHatch Fill	081h, 042h, 024h, 018h, 018h, 024h, 042h, 081h
9	Interleave Fill	0CCh, 033h, 0CCh, 033h, 0CCh, 033h, 0CCh, 033h
10	Wide Dot Fill	080h, 000h, 008h, 000h, 080h, 000h, 008h, 000h
11	Close Dot Fill	088h, 000h, 022h, 000h, 088h, 000h, 022h, 000h
0FFh	User is defining the pattern of the fill.	

In the case of a user-defined fill pattern, the register pair ES:BX point to 8 bytes of data arranged as a 8x8 bit pattern to be used for the fill pattern.

```
     DW     LINESTYLE   ; Set the line drawing pattern

  Input:
     AL      Line pattern number
     BX      User-defined line drawing pattern
     CX      Line width for drawing

  Return:
     Nothing
```


Sets the current line-drawing style and the width of the line. The line width is either one pixel or three pixels in width. The following table defines the default line styles:

Code	Description	16 Bit Pattern
AL = 0	Solid Line Style	1111111111111111B
AL = 1	Dotted Line	1100110011001100B
AL = 2	Center Line	1111110001111000B
AL = 3	Dashed line	1111100011111000B
AL = 4	User-defined line style	

If the value in AL is four, the user is defining a line style in the BX register. If the value in AL is not four, then the value in register BX is ignored.

```
        DW   TEXTSTYLE      ; Hardware text style control

  Input:
     AL      Hardware font number
     AH      Hardware font orientation
             0 = Normal,   1 = 90 Degree,   2 = Down
     BX      Desired X Character (size in graphics units)
     CX      Desired Y Character (size in graphics units)
```

Return:
BX Closest X Character size available (in graphics units)
CX Closest Y Character size available (in graphics units)

The TEXTSTYLE vector defines the attributes of the hardware font for output. The parameters affected are the hardware font to be used, the orientation of the font for output, the desired height and width of the font output. All subsequent text will be drawn using these attributes.

If the desired size is not supported by the current device, the closest available match to the desired size should be used. The return value from this function gives the dimensions of the font (in pixels) that will actually be used.

For example, if the desired font is 8x10 pixels and the device supports 8x8 and 16x16 fonts, the closest match will be the 8x8. The output of the function will be BX = 8, and CX = 8.

```
        DW      TEXT          ; Hardware text output at (CP)
```

Input:
ES:BX --> ASCII text of the string
CX The length (in characters) of the string.

This function sends hardware text to the output device. The text is output to the device beginning at the (CP). The (CP) is assumed to be at the upper left of the string.


```
        DW    TEXTSIZ      ; Determine the height and width of text
                           ; strings in graphics units.
```

Input:
ES:BX --> ASCII text of the string
CX The length (in characters) of the string.

```
  Return:
     BX       The width of the string in graphics units.
     CX       The height of the string in graphics units.

This function determines the actual physical length and width of a
text string.
The current text attributes (set by TEXTSTYLE) are used to determine
the actual
dimensions of a string without displaying it. The application can
thereby
determine how a specific string will fit and reduce or increase the
font size as
required. There is NO graphics output for this vector. If an error
occurs during
length calculation, the STAT field of the Device Status Record
should be marked
with the device error code.

        DW      FLOODFILL    ; Fill a bounded region using a flood
fill

  Input:
     AX         The x coordinate for the seed point
     BX         The y coordinate for the seed point
     CL         The boundary color for the Flood Fill

  Return:
     Nothing     (Errors are returned in Device Status STAT field).

This function is called to fill a bounded region on bitmap devices.
The (X,Y)
input coordinate is used as the seed point for the flood fill. (CP)
becomes the
seed point. The current FILLPATTERN is used to flood the region.

        DW      GETPIXEL       ; Read a pixel from the graphics
screen

  Input:
     AX         The x coordinate for the seed point
     BX         The y coordinate for the seed point

  Return:
     DL         The color index of the pixel read from the screen.

GETPIXEL reads the color index value of a single pixel from the
graphics screen.
The color index value is returned in the DL register.

        DW     PUTPIXEL     ; Write a pixel to the graphics screen

  Input:
     AX        The x coordinate for the seed point
     BX        The y coordinate for the seed point
     DL        The color index of the pixel read from the screen.
```

```
  Return:
     Nothing
```

PUTPIXEL writes a single pixel with the the color index value contained in the DL register.

```
        DW      BITMAPUTIL     ; Bitmap Utilities Function Table

  Input:
     Nothing

  Return:
     ES:BX      --> BitMap Utility Table.
```

The BITMAPUTIL vector loads a pointer into ES:BX, which is the base of a table defining special case-entry points used for pixel manipulation. These functions are currently only called by the ellipse emulation routines that are in the BGI Kernel. If the device driver does not use emulation for ellipses, this entry does not need to be implemented. This entry was provided because some hardware requires additional commands to enter and exit pixel mode, thus adding overhead to the GETPIXEL and SETPIXEL vectors. This overhead affected the drawing speed of the ellipse emulation routines. These entry points are provided so that the ellipse emulation routines can enter pixel mode, and remain in pixel mode for the duration of the ellipse-rendering process.

The format of the BITMAPUTIL table is as follows:

```
  DW  offset   GOTOGRAPHIC     ; Enter pixel mode on graphics hardware
  DW  offset   EXITGRAPHIC     ; Leave pixel mode on graphics hardware
  DW  offset   PUTPIXEL        ; Write a pixel to graphics hardware
  DW  offset   GETPIXEL        ; Read a pixel from graphics hardware
  DW  offset   GETPIXBYTE      ; Return a word containing pixel depth
  DW  offset   SET_DRAW_PAGE   ; Select page in which to draw primitives
  DW  offset   SET_VISUAL_PAGE ; Set the page to be displayed
```

```
  DW  offset   SET_WRITE_MODE  ; XOR Line Drawing Control
```

The parameters of these functions are as follows:

```
  GOTOGRAPHIC     ; Enter pixel mode on the graphics hardware
```

This function is used to enter the special Pixel Graphics mode.

```
  EXITGRAPHIC     ; Leave pixel mode on the graphics hardware
```

This function is used to leave the special Pixel Graphics mode.

```
  PUTPIXEL        ; Write a pixel to the graphics hardware
```

This function has the same format as the PUTPIXEL entry described previously.

```
  GETPIXEL        ; Read a pixel from the graphics hardware
```


This function has the same format as the GETPIXEL entry described previously.

```
  GETPIXBYTE      ; Return a word containing the pixel depth
```

This function returns the number of bits per pixel (color depth) of the graphics hardware in the AX register.

```
  SET_DRAW_PAGE   ; Select alternate output graphics pages (if any)
```

This function take the desired page number in the AL register and selects alternate graphics pages for output of graphics primitives.

```
  SET_VISUAL_PAGE ; Select the visible alternate graphics pages (if any)
```

This function take the desired page number in the AL register and selects alternate graphics for displaying on the screen.

```
  SET_WRITE_MODE ; XOR Line drawing mode control.
```

XOR Mode is selected if the value in AX is one, and disabled if the value in AX

is zero.

```
        DW      SAVEBITMAP   ; Write from screen memory to system
memory
```

Input:
ES:BX Points to the buffer in system memory to be written. ES:[BX] contains the width of the rectangle -1. ES:[BX+2] contains the heigth of the rectangle -1.

CX The upper left X coordinate of the rectangle.
DX The upper left Y coordinate of the rectangle.

Return:
Nothing

The SAVEBITMAP routine is a block copy routine that copies screen pixels from a defined rectangle as specified by (SI,DI) - (CX,DX) to the system memory.

```
        DW      RESTOREBITMAP    ; Write screen memory to the
screen.
```

Input:
ES:BX Points to the buffer in system memory to be read. ES:[BX] contains the width of the rectangle -1. ES:[BX+2] contains the heigth of the rectangle -1.

CX The upper left X coordinate of the rectangle.
DX The upper left Y coordinate of the rectangle.

AL The pixel operation to use when transferring the image into graphics memory. Write mode for block writing.
0: Overwrite mode
1: XOR mode

2: OR mode
3: AND mode
4: Complement mode

```
  Return:
     Nothing
```

The RESTOREBITMAP vector loads screen pixels from the system memory. The routine reads a stream of bytes from the system memory into the rectangle defined by (SI,DI) - (CX,DX). The value in the AL register defines the mode that is used for the write. The following table defines the values of the available write modes:

Pixel Operation	Code
Overwrite mode	0
Logical XOR	1
Logical OR	2
Logical AND	3
Complement	4

==========

```
       DW     SETCLIP         ; Define a clipping rectangle

  Input:
     AX        Upper Left X coordinate of clipping rectangle
     BX        Upper Left Y coordinate of clipping rectangle
     CX        Lower Right X coordinate of clipping rectangle
     DX        Lower Right Y coordinate of clipping rectangle

  Return:
     Nothing
```

The SETCLIP vector defines a rectangular clipping region on the screen. The registers (AX,BX) - (CX,DX) define the clipping region.

```
  DW       offset COLOR_QUERY    ; Device Color Information Query
```

This vector inquires about the color capabilities of a given piece of hardware. A function code is passed into the driver in AL. The following function codes are defined:

```
  >>> Color Table Size  AL = 000h
    Input:
       None:

    Return:
       BX    The size of the color lookup table.
       CX    The maximum color number allowed.
```

The COLOR TABLE SIZE query determines the maximum number of colors supported by the hardware. The value returned in the BX register is the number of color entries in the color lookup table. The value returned in the CX register is the highest number for a color value. This value is usually the value in BX minus one; however, there can be exceptions.

```
  >>> Default Color Table    AL = 001h
    Input:
       Nothing

    Return:
       ES:BX   --> default color table for the device
```

The DEFAULT COLOR TABLE function determines the color table values for the default (power-up) color table. The format of this table is a byte containing the number of valid entries, followed by the given number of bytes of color information.

Device driver construction particulars

The source code for a sample, albeit unusual, BGI device driver is included with this Toolkit to assist developers in creating their own. The demonstration driver is provided in two files, DEBVECT.ASM and DEBUG.C. This "Debug" driver doesn't actually draw graphics, but instead simply sends descriptive messages to the console screen (via DOS function call 9) upon receiving commands. Instead of simply playing back commands, your own driver would be structured similarly, but would access control ports and screen memory to perform each function.

Cookbook

1. Compile or assemble the files required.

2. Link the files together, making sure that the device vector table is the first module within the link.

3. Run EXETOBIN on the resulting .EXE or .COM file to produce a .BIN file. There should be no relocation fixups required.

4. Run program BH (provided with the toolkit) on the .BIN file to produce the .BGI file.

The resulting driver is now ready for testing. Examine the file TEST.C for an example of installing, loading, and calling a newly created device driver.

Examples

```
; To call any BGI function from assembly language, include the
  ; structure below and use the CALLBGI macro.

  CALLBGI MACRO   P
          MOV     SI,$&P                ; PUT OPCODE IN (SI)
          CALL    CS:DWORD PTR BGI_ADD  ; BGI_ADD POINTS TO DRIVER
          ENDM

  ; e.g., to draw a line from (10,15) to (200,300):

          MOV     AX, 10
          MOV     BX, 15
          MOV     CX, 200
          MOV     DX, 300
          CALLBGI VECT


  ; To index any item in the status table, include the status table
  ; structures below and use the BGISTAT macro.

  BGISTAT MACRO   P                     ; GET ES:<SI> --> BGI
```

```
STATUS
                        LES     SI, CS:DWORD PTR STABLE  ; GET LOCATION OF STATUS
TO SI
                        ADD     SI, $&P                  ; OFFSET TO CORRECT
LOCATION
                        ENDM

                ; e.g., to obtain the aspect ratio of a device:

                        BGISTAT ASPEC
                        MOV     AX, ES:[SI]              ; (AX)= Y/X *10000
```


CHAPTER

Borland Help system

This chapter defines the Borland Help system, including the source text file format, binary Help file format, and the run-time Help engine, all of which are necessary to support the following features:

Resizable Help display window.

Automatic wordwrapping during window resizing.

Smooth scrolling between logically connected Help screens.

Turbo Examples.

Free moving cursor.

How do I use it?

You can use the information provided in this chapter to write Help for your own products. The Help Linker (HL.EXE) is provided on the disk that accompanies this book. The Help files it produces are compatible with THELP.COM, a utility provided with most Borland compilers.

If you provide third-party libraries, you might want to offer reference material for those libraries in Borland Help so your customers can find information on your routines as easily as they do with Borland's own.

Wordwrap

The right margin for wrapping is based on the window width, and is independent of where the text is relative to the window. This means scrolling text horizontally through the window will not cause re-wrapping; only resizing the window causes re-wrap. The value specified in field leftMargin of the binary file File Header Record is also applied to the right edge of the window when determining the right margin for wrapping, but not for truncation of non-

wrapping text. Non-wrapping text is truncated at the physical right edge of the window.

Wrapping causes lines to move into and out of the display window at the bottom of the window only. It never affects lines above the wrapping line.

All hyphenated words in wrappable text must be removed from the Help source text. Here are the rules for wrapping at run time (breaking a line into two or more lines when the Help display window is too narrow to display the complete line):

For a line of Help text to be wrappable, it must begin with non-whitespace.

Wrapping only occurs at whitespace, and leaves whitespace behind at the end of the wrapping line.

For the purpose of wrapping, a keyword is treated as atomic, even if it contains whitespace.

A line isn't wrapped if only whitespace is truncated from the right to fit the current window width.

A line is truncated on the right (like nonwrapping text) if it doesn't contain whitespace that allows it to wrap.

Here are rules for converting hard returns to soft returns (allowing text to flow from the next line to fill the current line to the right margin):

A return at the end of a line that begins with whitespace is always hard.

If the next character following a return (first character of next line) is non-whitespace, then the return is soft; if the next character is whitespace, then

the return is hard.

These rules allow the existing Help text to wrap correctly with little or no change.

Smooth scroll within topics

All pages linked through the upContext and downContext fields of a keyword record are considered to be a single contiguous stream of text. Also, a single context (or screen) can contain any number of lines of text.

Turbo Example copy

A Turbo Example is a block of text in a Help screen that is set up for copying to the Clipboard. A single hot key copies the example to the Clipboard.

Only one Turbo Example is allowed per Help topic, where a topic is defined as the set of all contexts (screens, pages) joined through the upContext/downContext fields of a keyword record.

A Turbo Example is surrounded by ^E (0x05) characters in the context text.

Keywords cannot be nested in Turbo Examples and vice versa. The text of a Turbo Example can extend over several contexts (screens, pages), and can include both wrapping and non-wrapping text.

A special display attribute is defined to highlight Turbo Example text.

When copying a Turbo Example to the Clipboard, wrapping text is converted to fixed text by replacing soft returns with hard returns. The line in

the example text with the least amount of leading whitespace defines a left margin equivalent to this segment of leading whitespace. This left margin is deleted from all lines of the example text as it is copied to the Clipboard. Trailing whitespace is also deleted from all lines. For example, if the Turbo Example text is

```
" void main( void ) { "
" printf( "Hello world\n" ); "
" }"
```

this is what gets copied to the Clipboard:

```
"void main( void ) {"
"printf( "Hello world\n" );"
"}"
```

Summary of keyboard and mouse interaction

Following is a summary of keyboard and mouse usage supported by the run-time engine while the Help window is active.

UpArrow

Moves cursor up one row in current column. If the cursor is already at the top of the window, scroll the text down one row in the window; if the cursor is at the top of the topic text, ignore the command.

DownArrow

Moves cursor down one row in current column. If the cursor is already at the bottom of the window, scroll the text up one row in the window; if at the bottom of the topic text, ignore the command.

LeftArrow

Moves cursor left one column on current row. If the cursor is already at the left edge of the window, scroll the text right horizontally by one column; if at the left edge of the topic text, ignore the command.

RightArrow

Moves cursor right one column on current row. If the cursor is already at the right edge of the window, scroll the text left horizontally by one column. The text can be scrolled left until column MaxHelpColumn is in the rightmost column of the Help window.

CtrlLeftArrow

Moves cursor left to the start of the previous word. A word is defined as a sequence of any of the following characters: (a..z), (A..Z), (0..9), or (_, $, #). If no further words remain on the current row, look for the word starting at the end of the previous row; if there's no previous row, ignore the command. Scroll the text in the window as necessary to keep the cursor in the window.

CtrlRightArrow

Like Ctrl Left, except moves the cursor right to the start of the next word.

Home

Moves cursor to first non-whitespace character of current row, scrolling the topic text horizontally in the window if necessary; if the row is all whitespace, move to column 1.

End

Moves cursor to one column past last non-whitespace character of current row, scrolling the topic text horizontally in the window if necessary.

PgUp

Scrolls topic text down in the window by the number of lines displayable in the window, or by the number of lines remaining to the top of the topic text, whichever is less. The cursor position is not affected.

PgDn

Scrolls topic text up in the window by the number of lines displayable in the window, or by the number of lines remaining to the bottom of the topic text, whichever is less. The cursor position is not affected.

Shift

If the Shift key is held down, and one or more sequences of the previous cursor control keys are pressed, a block of Help text will be selected. The block includes the character position at which the cursor was originally positioned, up to but not including the final resting position of the cursor. The block is

highlighted as the cursor is moved. The block remains in effect until a cursor control key is pressed without the Shift key, or until it is copied to the Clipboard.

Tab

Selects the next keyword in the current topic text. If the last keyword in the topic is currently selected, then selects the first keyword in the topic. If there are no keywords in the topic, ignores the command. If the next keyword is not currently displayed in the window, scrolls the window horizontally and/or vertically to place the keyword text just inside the window.

ShiftTab

Like Tab, except selects previous keyword.

Enter

If a selected keyword is currently displayed in the Help window, switch to its context. If no keyword is displayed (even though one or more exist

elsewhere in
the topic text), ignore the command.

Any other key is used for incremental searching between keywords in
the topic
text.

clicking
Clicking moves the cursor to the mouse cursor position, and cancels
selected
text, if any. If the mouse cursor is on a keyword, the keyword
becomes the
active keyword.

Shift
clicking
Shift+clicking causes the current block of selected text to be
extended to the
cursor position.

double clicking
Double clicking moves the cursor to the mouse cursor position, and
cancels
selected text, if any.

If the cursor is not positioned on a keyword, then do an index
search for the
token the cursor is currently positioned on, and if a match is
found, switch
contexts. If the cursor is on a keyword, switch to the keyword's
context.

A "token" is defined the same as a word for cursor movements (see
the
description of Ctrl-Left.)

right button
No action is defined for the right mouse button in the Help window.

dragging

Dragging the mouse in the Help window is equivalent to moving the
cursor with
the arrow keys while depressing the Shift key; that is, it selects

text while
allowing horizontal and vertical scrolling.

Scroll bars
Scroll bars are supported in the usual manner for scrolling Help topic text
within the window.

F1

Switches to context specified by mainIndexScreen field of File Header Record.

AltF1

If previous context recorded, switch to previous context, else switch to
mainIndexScreen context.

CtrlF1

If the cursor is not positioned on a keyword, then does index search for the
token the cursor is currently positioned on and, if a match is found, switches
contexts.

If the cursor is on a keyword, switches to the keyword's context.

A "token" is defined the same as a word for cursor movements (see description of
Ctrl Left).

Esc

Closes the Help window.

Menu options
Two Edit menu options apply when Help is active:

Copy copies the current selected text from the topic text to the Clipboard. If
no text is currently selected, the command is disabled (grayed in the menu). The
text is "unselected" after the copy. The rules for coercing text during a Turbo
Example copy (noted earlier), also apply during a generalized copy to Clipboard.

Copy Example copies the Turbo Example text from the current topic text, if any,
to the Clipboard. If the current topic has no Turbo Example, the command is
disabled (grayed in the menu).

Incremental searching

Incremental searching is supported for movement between keywords in topic text.

Literal characters entered at the keyboard are matched against successive characters in the text of keywords, and the selected keyword is changed based on the characters entered. Backspace strips successive characters from the match string. Explicit cursor movements cancel the incremental search.

Index context

A special context code (;INDEX) is recognized by the Help system that maps onto an internally generated topic. The topic consists of all entries in the index table of the Help file; index entries are stored as keywords. The user can then use any of the normal means of moving between these index keywords, and switch to contexts referenced in the index table.

Creating online Help text

First and foremost rule: Any command that you use in the Help file must be immediately preceded by a semicolon (;). Letter case does not matter unless you're using the ;CASESENSE command.

Second rule: You must put hard returns at the end of your lines.

There are several (optional) initial setup commands that you can place at the beginning of your Help files.

;CASESENSE causes Help index entries and screen names to be case sensitive.

;STAMP places (a usually human readable) ID stamp in the Help file to identify file it as Help file.

;SIGNATURE places another ID stamp in the Help file.

;VERSION codes a version number into the Help file.

Recommended practice is to include any of these setup commands into a separate file and always include that file first when you create Help.

An example

Here is an example of the typical commands you'd use in a single Help screen format:

```
;COMMENT I can place this here; it won't appear
;COMMENT when you bring up the Help file
;SCREEN waditdo

Turbo Dictionary

When you select one of the items on this
menu, you can learn everything you've ever
wanted to know about it until you think
you're going to implode with knowledge. Your
choices include:

Note that these "^B"s are the actual ^B character (0x02).

 ^BAnnouncer     ^B  ^BArchitect     ^B



 ^BGame show host^B  ^BPlumber       ^B

You'll want to use this command after a
particularly long night of partying when
you need something titillating to keep you
awake or possibly to fool some higher-up
into thinking that you're really working.

;KEYWORD don
;KEYWORD art
;KEYWORD dailydouble
;KEYWORD potpourri
;INDEX Dictionary
;ENDSCREEN
```

Here's an explanation of each command used in the previous example:

;COMMENT

;COMMENT is an optional command you can use when you want to make a note to yourself (or anyone else reading the file) about that particular Help screen (or anything else for that matter). There's no limit to how many ;COMMENTs you can put in a file. You can also use ;COMMENT to keep track of modifications and authors. Naturally, comment text doesn't appear in the final Help file.

;SCREEN

;SCREEN marks the beginning of each new Help screen. The ;SCREEN name given in this command names the screen that Help searches for when the user selects a keyword. (See the ;KEYWORD command, below.)

;KEYWORD

;KEYWORD is an optional command that defines which Help screen to bring up when the user selects the matching keyword. Basically, the associated keyword is a reference. Perhaps a better way to put it is to compare it with a similar use in an encyclopedia or thesaurus. In defining or explaining an entry, these reference books may highlight or capitalize other related entries, or tell you to See other related entry.

When the user calls up Help, all keywords appear highlighted. You can move around the keywords using the Up arrow, Down arrow, Right arrow, and Left arrow keys. The keyword you're positioned on is highlighted; to select it, press Enter.

Here's another example:

```
;SCREEN metaphysics

 Metaphysics

Metaphysics is a branch of philosophy concerned
with the ultimate nature of existence. Ontology
(the study of the nature of being), cosmology,
and philosophical theology are usually considered
```

```
its main branches. The term comes from the
metaphysical treatises of Aristotle, who presented
the First Philosophy (as he called it) after the
Physics.

 See also
^B  Kant     ^B
^B  Fichte   ^B
^B  Schelling^B
^B  Hegel    ^B

;KEYWORD kant
;KEYWORD fichte
;KEYWORD schelling
;KEYWORD hegel
;INDEX Metaphysics
;ENDSCREEN

;SCREEN kant

Kant  1724-1804

German philosopher, one of the greatest
figures in the history of ^Bmetaphysics^B.
Kant proposed that objective reality is known
only insofar as it conforms to the essential
structure of the knowing mind. Only objects
of experience (phenomena) may be known, where
things lying beyond experience (noumena) are
unknowable, even though in some cases we
assume a prior knowledge of them. The existence
of such unknowable "things-in-themselves" can
be neither confirmed nor denied, nor can they
be scientifically demonstrated.

;KEYWORD metaphysics
;INDEX Kant
;ENDSCREEN
```

Notice that screen metaphysics has four keywords: Kant, Fichte, Schelling, and Hegel. For the sake of brevity, only one screen connected to metaphysics has been shown--screen Kant.

Note that we showed the ^B's as two separate characters, but they should actually be the ^B character: 0x02.

Each keyword within the screen text is delimited by ^B's and has a matching

;KEYWORD command. (So the Help Linker knows which screen a given keyword is to bring up when selected.) Read the following section, "More about ^B's" for further explanation.

This example shows the keywords formatted as a single column (which will wrap to multiple columns when the Help window is wide enough). You can also use keywords within the text of a paragraph.

Whatever the keyword happens to be, your beginning and ending ^B's must be on the same line; the Help Linker gives an error if you try to wrap a keyword on two lines.

;ENDSCREEN

;ENDSCREEN ends the screen you began with ;SCREEN; there's no argument necessary.

;PAGE

;PAGE is a linking command between two or more Help screens of related information. Pressing PgUp takes you to the next screen; PgDn takes you to the previous screen. A good example of ;PAGE can be found on disk.

Compiling and linking online Help

Help linker command line syntax:

```
hl {inputFile | @respFile} [/ooutFile] [/eerrorLimit] [/x]
```

where

[p]	means p is optional.
{p}	means zero or more repetitions of p.
p|q	means choose p or q.

Parameters can appear in any order.

inputFile The name of a Help text file--any command line parameter not beginning with a "/" is assumed to be an input file specification, and any number can appear on the command line. If no path is specified, the file is taken from the current directory.

@respFile respFile is the path/name of a response file containing the names of Help text input files. The file can specify any number of input files. Each file should be listed on a separate line in the file. Lines beginning with a semi-colon (;) are ignored and can be used for comments. If no path is specified, the file is taken from the current directory. Any number of response files can be specified on the command line; however, response files can not be nested.

Note

DOS file wildcards can be used in any inputFile specification, either on the command line or in a response file.

/ooutFile outFile is the path/name of the file into which the compiled Help data is to be stored. If this parameter is missing, the data is stored in TCHELP.TCH in the current DOS work directory.

/eerrorLimit errorLimit is the number of errors that need to be detected before the Help Linker will terminate without completing the link operation. If the parameter is missing, the Linker will terminate on any error.

/x

If this switch is present, the Help Linker will not automatically create and insert an index table screen in the resulting binary Help file. Since THELP automatically creates an index screen "on-the-fly," not including the /x switch will only result in a larger Help file.

Binary Help file format

The Binary Help File is comprised of a sequence of records. All records are mandatory, and the sequence of the records is significant.

The records of the file are grouped into four major sections as follows:

Administrative
File Stamp

File Signature

File Version

File Header Record

Compression Record

Context Table

Index Table

Context Descriptions: A series of 1 or more pairs of records:
Text Record

Keyword Record

The administrative records help to identify the file as a valid Help file, and provide information necessary to interpret the remaining records of the file.

The Context Table is a table defining every individually addressable "chunk" of Help text. Each Context is given a unique identification number which happens to be a direct index into the Context Table. The indexed element of the table gives an absolute offset into the Help file where a complete description of the context can be found.

The Index Table is a sorted list of text labels, each with an associated Context Number. The Index Table allows Contexts to be referenced via a text label.

The fourth and final area is the Context Descriptions. This is a list of one or more pairs of Text and Keyword Records. The Text Records give the actual text associated with each context, and they are directly addressed by the elements of the Context Table. All Text Records have an associated Keyword Record which defines linkage to other Contexts, as well as cross reference keywords embedded in the context text.

Each file record type is described in detail in the remaining sections of this document.

In the following sections, assume the following definitions:

```
typedef unsigned char byte;
typedef unsigned short word;
```

File Stamp

An ASCIIZ string identifying the file in "human readable" terms. For example, the following strings are used in Turbo C++ and Turbo Pascal respectively:

TURBO C Help FILE.\0

TURBO PASCAL Help FILE.\0

The terminating null character is followed by a DOS End-of-File character (0x1A), so that a user attempting to "TYPE" the Help file under DOS will simply see the File Stamp string displayed.

The text of this string is defined using the ;STAMP command in Help source text processed by the Help Linker.

File Signature

An ASCIIZ string helps to further identify a file as a valid Borland

Help file.

The string may be any value mutually agreed between the author of the Help text, and the programmer of the run-time code. The value currently used by Borland language products is:

```
$*$* &&&&$*$
```

The text of this string is defined by the ;SIGNATURE command in Help source text processed by the Help Linker.

File Version

Two bytes that define the version of the Help Format, and of the Help File Text, respectively:

```
typedef struct
{
    byte    formatVersion;
    byte    textVersion;
```



```
} TPversionRec;
```

formatVersion defines the version of the Help file format. It allows the run-time code to test that its reader is capable of reading the Help file. This version code is hard-coded into both the Help Linker and the run-time code, and is updated when the file format is revised. The format defined in this document requires that field formatVersion be set to 52.

Field textVersion defines the version of the text (i.e contents) of the Help file. The value is set using command ;VERSION in the Help source text processed by the Help Linker. The run-time code of Borland language products currently ignore this value.

Record Headers

The remaining records of a Help file have a common format which includes a header identifying the record's type and its length:

```
typedef struct
{
   byte    recType;
   word    recLength;
} TPrecHdr;
```

Field recType is a code which identifies the record type. The following record types are currently defined, and each is explained in further detail in the sections which follow:

```
enum {
   RT_FileHeader   = 0,
   RT_Context      = 1,
   RT_Text         = 2,
   RT_Keyword      = 3,
   RT_Index        = 4,
   RT_Compression  = 5
};
```

Field recLength gives the length of the contents of the record in bytes, not including the record header. The contents begin with the first byte following the header.

Note that while this record structure allows for an arbitrary ordering of records within the file, the existing Borland language products assume a fixed record ordering, which is the same order used to describe the records in the following sections.

File Header Record

Defines various parameters and options common to the entire Help file.

```
typedef struct
{
   word    options;
```

```
    word    mainIndexScreen;
    word    maxScreenSize;
    byte    height;
    byte    width;
    byte    leftMargin;
  } TPfileHdrRec;
```

options

options is a bitmapped field that let you select various options. Only one is currently supported.

OF_CaseSense (0x0004)

If set, index tokens are listed in mixed case in the Index Record, and index searches should be case sensitive.

If cleared, index tokens are all uppercase in the Index Record, and index searches should ignore case.

Set by ;CASESENSE command in Help source text processed by the Help Linker.

mainIndexScreen

The context number of the context designated by the ;MAININDEX command in the Help source text processed by the Help Linker. If ;MAININDEX wasn't used, mainIndexScreen is set to zero.

maxScreenSize

The number of bytes in the longest Text Record in the file (not including its header). This field is not currently used.

height, width

The default size in rows and columns, respectively, of the display area of a Help window.

Set using the ;HEIGHT and ;WIDTH commands in Help source text processed by the Help Linker.

leftMargin

The number of columns to leave blank on the left edge of all rows of Help text displayed.

Set using the ;LMARGIN command in Help source text processed by the Help Linker.

Compression Record

Defines how the contents of Text Records are encoded. The record has the following general form:

```
typedef struct
{
    byte    compType;
    byte    charTable[ 14 ];
```



```
} TPcompRec;
```

compType is a code that identifies the type of compression used. Nibble encoding (CT_Nibble) is the only compression method currently supported.

```
enum {
    CT_Nibble       = 2
};
```

The text of a Text Record is encoded as a stream of nibbles. The nibbles are stored sequentially in the bytes of the text record; the low nibble of a byte logically precedes the high nibble of the byte in the nibble stream.

Nibble values (0x0...0xD) are direct indexes into the charTable field of the Compression Record. The indexed entry is the literal character represented by the nibble. Obviously, the Help Linker chooses the 14 most frequent characters for inclusion in this table. One exception is that element 0 of this table always maps to a byte value of 0.

The remaining two nibble values have special meanings:

```
enum {
    NC_RawChar  = 0xF,
    NC_RepChar  = 0xE
};
```

Nibble code NC_RawChar introduces two additional nibbles which define a literal character; the least significant nibble appears first.

Nibble code NC_RepChar defines a repeated sequence of a single character. The next nibble gives the repeat count less two (i.e. counts from 2 to 17 are possible). The next nibbles define the character to repeat; the repeat character may be either a single nibble in the range (0x0 .. 0xD) representing an index into charTable, or it may be represented by a three nibble NC_RawChar sequence.

Context table

A table of absolute file offsets which relates Help contexts with their associated text. The first word of the record gives the number of contexts in the table.

The remainder of the record is a table of n (n given by first word) 3-byte integers (LSByte first). The table is indexed by context number (0 to n-1). The 3-byte integer at a given index is an absolute byte offset in the Help file where the text of the associated context begins.

The 3 byte integer is signed (2's complement). Two special values are defined:

-1 Use Index Screen text - defined in File Header Record.

-2 No Help is available for this context.

Context Table entry 0 is not used.

Index table

A list of index descriptors.

An index is a token (normally a word or name) that has been explicitly

associated with a context using the ;INDEX command in the source text processed by the Help Linker. More than one index may be associated with a context, but any given index can not be associated with more than one context.

The list of index descriptors in the Index Record allows the text of an index token to be mapped into its associated context number.

The first word of the record gives the number of indexes defined in the record.

The remaining bytes of the record are grouped into index descriptors. The descriptors are listed in ascending order based on the text of the index token (normal ASCII collating sequence). If the OF_CaseSense flag is not set in the option field of the File Header Record, all indexes are in uppercase only.

Each index descriptor is of the following form:

```
byte    lengthCode;
byte    uniqueChars[ 1 .. n ];
word    contextNumber;
```

The bits of lengthCode are divided into two bit fields. Bits (7..5) specify the number of characters to carry over from the start of the previous index token string. Bits (4..0) specify the number of unique characters to add to the end of the inherited characters. Field uniqueChars gives the n unique characters to add.

For example, if the previous index token was addition, and the next index token is advanced, we would inherit two characters from the previous token (ad), and add six unique characters (vanced); thus, lengthCode would be 0x46.

contextNumber gives the context number of the context associated with the index. This number is an index into the Context Table described on page 133.

Text Record

Defines the compressed text of a context.

Text Records and Keyword Records (see 134) appear in pairs; one pair for each context in the Help file. The Text Record always precedes its associated Keyword Record. Text Records are addressed in the Help file through file offset values

found in the Context Table.

The recLength field of the Text Record's header defines the number of bytes of compressed text in the record. The Compression Record defines how the text is compressed. If the text record is nibble encoded, and the last nibble of the last byte is not used, it is set to 0 - this translates to a 0 byte when the text is decoded, and the 0 byte represents a blank line.

Lines of text comprising the Text Record are stored as ASCIIZ strings.

Keyword Record

Defines keywords embedded in the preceding Text Record, and identifies related Text Records.

The record begins with the following fixed fields:

```
word upContext;
word downContext;
word keywordCnt;
```

upContext and downContext give the context numbers of the previous and next sections of text in a sequence, respectively. Either may be zero, indicating the end of the context chain.

keywordCnt gives the number of keywords encoded in the associated Text Record. Immediately following this field is an array of keywordCnt Keyword Descriptor Records of the following form:

```
typedef struct
{
    word    kwContext;
} TPkwDesc;
```

The keywords in a Text Record are numbered from 1 to keywordCnt in the order they appear in the text (reading left to right, top to bottom).

kwContext is a context number (index into the Context Table) indicating which context to switch to if this keyword is selected by the user.

CONTENTS

Introduction 1
Why open architecture? 1
Borland language tools 2
How to use this book 2
Tools discussed 2
Accompanying software 3
Dynamically dispatchable virtual tables 23
Chapter 2 Object file contents 25
Turbo object file comment

A brief disclaimer 3

Chapter 1 C++ object mapping 5

Nonstatic data members 5
Nonvirtual base classes 5

Virtual base classes 6
Empty classes 10

Addressing of class instances and this 10

Virtual table pointers . . . 10

Virtual tables 11

Virtual function calls, virtual thunks 11

Calling conventions for member functions 11

Pointers to class members . . 12

Pointers to data members . 12

Pointers to function members 13

Static data members 14
_export classes 14

Passing classes by value . . 14

Initialization and finalization of nonlocal static objects . 14

Conventions for constructors and destructors 14

Constructors 14

Destructors 15

RTL helper functions 15

Name mangling 18
Encoding of nested and template classes 19
Encoding of function names . 19

records 26

0x00 Compiler identification 26

0xe0 External symbol type index 26

0xe1 Public symbol type index 27

0xe2 Structure member definition 27

0xe3 Type definition . . . 29

Simple types 32

Pascal string type 32

TID_PSTR 32

Labels 32

TID_LABEL 32

Integral range types . . . 32

Cobol-style BCD 33

TID_BCDCOB 33

Pointer types 33

TID_NEAR and TID_NEAR386 33

TID_FAR and TID_FAR386 . 33

TID_SEG 34

TID_NREF 34

TID_FREF 34

Array types 34

TID_CARRAY 34

TID_VLARRAY 34

TID_PARRAY 34

Very large structure types 35

TID_VLSTRUCT and TID_VLUNION 35

Ordinary functions . . . 19
Constructors, destructors, and overloaded operators . 20
Type conversions 21
Encoding of arguments . . . 21
Enumerated types . . . . . TID_ENUM and TID_PENUM .
Function types . . . . . TID_FUNCTION . . . . . .
Sets . . . . . TID_SET . . . . . .

Binary files 36
TID_BFILE 36
Member/duplicate functions 36
TID_SPECIALFUNC 36
C++ Class 36
TID_CLASS 37
Pointed-to members . . . 37
TID_MEMBERPTR 37
New style pointed-to members 37
TID_NEWMEMBERPTR . . . 37
0xe4 Enum member definition 37
0xe5 Begin scope record . 38
0xe6 Locals definition record 38
SC_TYPEDEF (6) and SC_TAG (7) 38
0xf9 Debug Information Version . . . . . .
0xfa Module optimization flags . . . . . .
.OBJ extensions for 32 bits .
VIRDEF Records . . . . . .
Chapter 3 Symbol table format
Symbols . . . . . .
Modules . . . . . .
Source files . . . . . .
Line numbers . . . . . .
Scopes . . . . . .
Segments . . . . . .
Segment/source file correlations . . . . . .
Types . . . . . .
Simple types and common fields . . . . . .

SC_STATIC (0) 39
63
SC_ABSOLUTE (1) 39
63
SC_AUTO (2) and SC_PASVAR
64
(3) 39
64
SC_REGISTER (4) 39
64
SC_CONST (5) 39
SC_OPT (8) 40
65
SC_AUTO and SC_PASVAR . 40
65
SC_REGISTER 40
0xe7 End of scope 41
65
0xe8 Select source file . 41
0xe9 Dependency file
65
definition 41
66
0xea Compile parameters
66
record 42
66
0xeb External symbol matched
66
type index 43
66
0xec Public symbol matched
type index 43
67
0xed Class definition . . 44
Class descriptions . . . 44
67
0xee Coverage offset
68
record 45
68
0xf5 Begin large scope
record 45
69
0xf6 Large offset locals
75
definition record 46
SC_STATIC (0) 46
75
SC_ABSOLUTE (1) 46
76
SC_AUTO (2) and SC_PASVAR
78
(3) 47
78
0xf7 Large end of scope . 47
78
0xf8 Member function . . . 47
79
Pascal strings (12 bytes) .
Ranges (24 bytes) . . . . .
BCD COBOL (12 bytes) . . . .
Pointers (12 bytes) . . . .
C arrays (12 bytes) . . . .
Very large arrays (12
bytes) . . . . . .
Pascal arrays (24 bytes) . .
Structs and unions (12
bytes) . . . . . .
Very large structs and unions
(24 bytes) . . . . . .
Enums (24 bytes) . . . . . .
Functions (12 bytes) . . . .
Labels (12 bytes) . . . . .
Sets (12 bytes) . . . . . .
Binary files (12 bytes) . .
Function prototypes
(24 bytes) . . . . . .
Special functions (24
bytes) . . . . . .
Classes (12 bytes) . . . . .
Member pointers (24 bytes) .
Near and far references
(24 bytes) . . . . . .
Members . . . . . .
Structure and union
members . . . . . .
Class table . . . . . .
Special cases . . . . . .
Parent table . . . . . .
Scope class table . . . . . .
Module class table . . . . . .

Coverage offset map table . . 79
Coverage offset table 80
Browser definition table . . 80
Optimized symbol table . . . 80
Module Optimization Flags Table,
Reference Information Table . 81
Names 82
Debugging Turbo Pascal
overlays 82

Chapter 4 Project file format 85
Project file utilities . . . 85
How the utilities work . . 85
Using the examples 86
Show overview (-o) . . . 87
Show modules (-p) 87
Show modules with dependencies (-P) 87
Show options (-t) 87
Set options (-s) 87
Show note (-n) 87
Show header (-h) 87
TRANCOPY syntax 87
STRIPPRJ syntax 88
Format of the Project file . 88
Header Section . . . . . .
The driver status table . .
The device driver vector table . . . . . .
Vector Descriptions . . . .
Device driver construction particulars . . . . . .
Cookbook . . . . . .
Examples . . . . . .

Chapter 6 Borland Help system
How do I use it? . . . . . .
Wordwrap . . . . . .
Smooth scroll within topics .
Turbo Example copy . . . . .
Summary of keyboard and mouse interaction . . . . . .
Menu options . . . . . .
Incremental searching . . .
Index context . . . . . .
Creating online Help text .
An example . . . . . .
;COMMENT . . . . . .
;SCREEN . . . . . .

Header information 89
126
Sections in the project
128
file 89
128
Block Type 50--Options
section 90
128
Block Type 51--Header
129
section 90
130
Block Type 10--Transfer
130
section 92
130
Block Type 52--Note
131
section 92
131
Block Type 53--Module
132
section 93
132
Block Type 54--Dependency
132
section 94
132
Block Type 55--Extension
132
section 98
132

Chapter 5 The BGI driver
133
toolkit 101
133
BGI run-time architecture . . 101
134
BGI Graphics Model 102
135

;KEYWORD . . . . . .
;ENDSCREEN . . . . . .
;PAGE . . . . . .
Compiling and linking online
Help . . . . . .
Binary Help file format . .
File Stamp . . . . . .
File Signature . . . . . .
File Version . . . . . .
Record Headers . . . . . .
File Header Record . . . .
options . . . . . .
OF_CaseSense (0x0004) .
mainIndexScreen . . . .
maxScreenSize . . . . .
height, width . . . . .
leftMargin . . . . . .
Compression Record . . . .
Context table . . . . . .
Index table . . . . . .
Text Record . . . . . .
Keyword Record . . . . . .